

C++设计模式（全23种）

设计模式概述

设计模式是软件开发人员在软件开发过程中面临的一般问题的解决方案。这些解决方案是众多软件开发人员经过相当长的一段时间的试验和错误总结出来的解决某一类问题的一种编码方案。

文章目录

设计模式概述

设计模式特点

一、创建型模式

- Factory 模式（工厂模式）
- AbstactFactory 模式（抽象工厂模式）
- Singleton 模式（单例模式）
- Builder 模式（建造者模式）
- Prototype 模式（原型模式）

二、结构型模式

- Bridge 模式（桥接模式）
- Adapter 模式（适配器模式）
- Decorator 模式（装饰器模式）
- Composite 模式（组合实体模式）
- Flyweight 模式（享元模式）
- Facade 模式（外观模式）
- Proxy 模式（代理模式）

三、行为模式

- Template 模式（模板模式）
- Strategy 模式（策略模式）
- State 模式（状态模式）
- Observer 模式(观察者模式)
- Memento 模式（备忘录模式）
- Mediator 模式（中介者模式）
- Command 模式（命令模式）
- Visitor 模式（访问者模式）
- Chain of Responsibility 模式（责任链模式）
- Iterator 模式（迭代器模式）
- Interpreter 模式（解释器模式）

引用

设计模式特点

类型	设计模式	描述
创建型	Factory 模式	被实例化的子类
	AbstactFactory 模式	产品对象家族
	Singleton 模式	针对一个类的唯一实例
	Builder 模式	如何创建一个组合对象
	Prototype 模式	针对被实例化的类
结构型	Bridge 模式	对象的实现

	Adapter 模式	针对对象的接口
	Decorator 模式	对对象的职责，不生成子类
	Composite 模式	一个对象的结构和组成
	Flyweight 模式	对象的存储开销
	Facade 模式	对一个子系统的接口
	Proxy 模式	如何访问一个对象；该对象的位置
行为型	Template 模式	对算法中的某些步骤
	Strategy 模式	算法
	State 模式	对象的状态
	Observer 模式	对多个对象依赖于另外一个对象，而这些对象又如何保持一致
	Memento 模式	对一个对象中哪些私有信息存放在该对象之外，以及在对什么时候进行存储
	Mediator 模式	对象间怎样交互、和谁交互
	Command 模式	何时、怎样满足一个请求
	Visitor 模式	某些可作用于一个（组）对象上的操作，但不修改这些对象的类
	Chain of Responsibility 模式	满足一个请求的对象链
	Iterator 模式	如何遍历、访问一个聚合的各元素
	Interpreter 模式	对一个语言的文法及解释

一、创建型模式

Factory 模式（工厂模式）

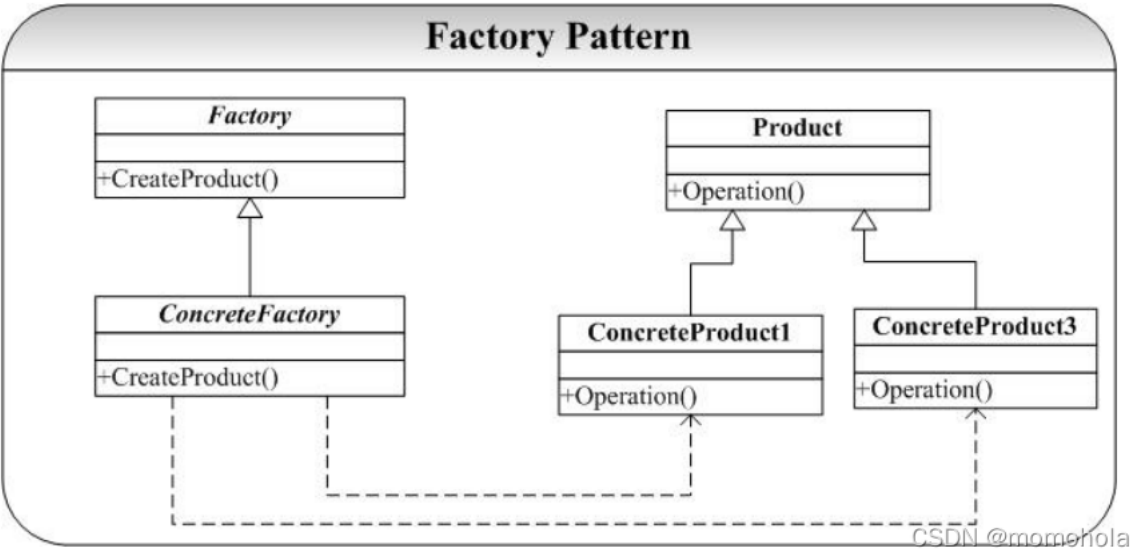
在面向对象系统设计中经常可以遇到以下的两类问题：

- 我们经常会抽象出一些类的公共接口以形成抽象基类或者接口。这样我们可以通过声明一个指向基类的指针来指向实际的子类实现，达到了多态的目的。所以就不得不在要用到子类的地方写new 对象。这样实体类的使用者必须知道实际的子类名称，以及会使程序的扩展性和维护变得越来越困难。
- 还有一种情况就是在父类中并不知道具体要实例化哪一个具体的子类。只能在父类中写方法调用，具体调用哪一个类的方法交给子类实现。

以上两个问题也就引出了 Factory 模式的两个最重要的功能：

- 1) 定义创建对象的接口，封装了对象的创建。
- 2) 使得具体化类的工作延迟到了子类中。

举例：



代码:

```
1 //Product.h
2 #ifndef _PRODUCT_H_
3 #define _PRODUCT_H_
4 class Product
5 {
6 public:
7 virtual ~Product() = 0;
8
9 protected:
10 Product();
11 private:
12 };
13 class ConcreteProduct:public Product
14 {
15 public:
16 ~ConcreteProduct();
17 ConcreteProduct();
18
19 protected:
20 private:
21 };
22 #endif //~_PRODUCT_H_
23
24
25 //Product.cpp
26 #include "Product.h"
27 #include <iostream>
28 using namespace std;
29 Product::Product()
30 {
31
32 }
33 Product::~~Product()
34 {
35 }
36 ConcreteProduct::ConcreteProduct()
37 {
38 cout<<"ConcreteProduct..."<<endl;
39 }
40 ConcreteProduct::~~ConcreteProduct()
41 {
42 }
43
44
45 //Factory.h
46 #ifndef _FACTORY_H_
47 #define _FACTORY_H_
48 class Product;
49 class Factory
50 {
51 public:
52 virtual ~Factory() = 0;
53 virtual Product* CreateProduct() = 0;
54 protected:
55 Factory();
56 private:
57 class ConcreteFactory:public Factory
58 {
59 public:
60 ~ConcreteFactory();
61 ConcreteFactory();
62 Product* CreateProduct();
63 protected:
64 private:
65 };
66 #endif //~_FACTORY_H_
67
68 //Factory.cpp
69 #include "Factory.h"
```

```

70 | #include "Product.h"
71 | #include <iostream>
72 | using namespace std;
73 | Factory::Factory()
74 | {
75 | }
76 | Factory::~~Factory()
77 | {
78 | }
79 | ConcreteFactory::ConcreteFactory()
80 | {
81 |     cout<<"ConcreteFactory....."<<endl;
82 | }
83 | ConcreteFactory::~~ConcreteFactory()
84 | {
85 | }
86 | Product* ConcreteFactory::CreateProduct()
87 | {
88 |     return new ConcreteProduct();
89 | }
90 |
91 | //main.cpp
92 | #include "Factory.h"
93 | #include "Product.h"
94 | #include <iostream>
95 | using namespace std;
96 | int main(int argc, char* argv[])
97 | {
98 |     Factory* fac = new ConcreteFactory();
99 |     Product* p = fac->CreateProduct();
100 |     return 0;
101 | }

```

总结:

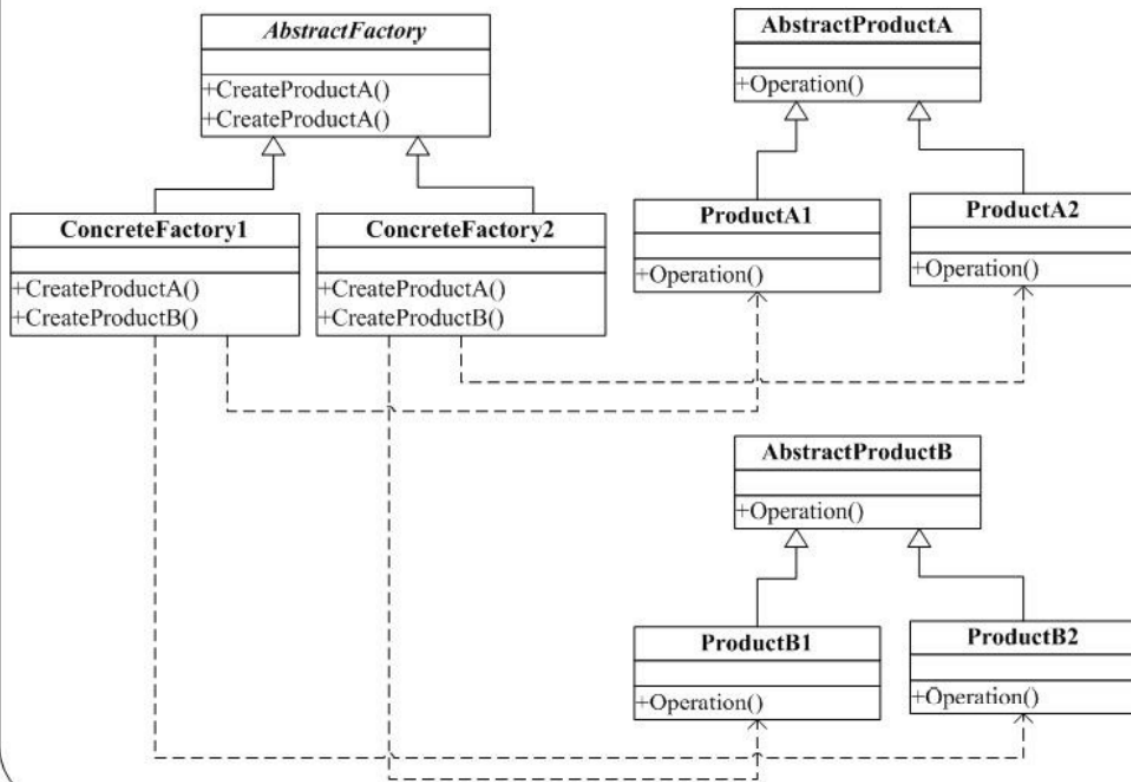
- 如果为每一个具体的 ConcreteProduct 类的实例化提供一个函数体，那么我们可能不得不在系统中添加了一个方法来处理这个新建的 ConcreteProduct
- 可以看出，Factory 模式对于对象的创建给予开发人员提供了很好的实现策略，但是Factory 模式仅仅局限于一类类（就是说 Product 是一类，有一个共同的基类），如果我们要为不同类的类提供一个对象创建的接口，那就要用 AbstractFactory 了。

AbstractFactory 模式（抽象工厂模式）

假如我们要买水果，水果的产地来自中国、日本、美国，每个国家的水果种类都可以分为苹果、香蕉、梨子。作为开发者，我们就不得不创建苹果类（香蕉和梨子类似），然后每种苹果都继承自苹果类。每上架一个国家的苹果我们都要实现一次苹果类，这样就会有成千上万的苹果类需要被创建，AbstractFactory 模式就是用来解决这类问题的：要创建一组相关或者相互依赖的对象。

举例：

AbstractFactory Pattern



CSDN@momohola

代码:

```

1 //抽象工厂模式
2 #include <iostream>
3 using namespace std;
4
5 //苹果的抽象
6 class AbstractApple {
7 public:
8     virtual void showName() = 0;
9 };
10
11 //中国苹果
12 class ChinaApple :public AbstractApple {
13 public:
14     virtual void showName() {
15         cout << "中国苹果" << endl;
16     }
17 };
18
19 //美国苹果
20 class USAApple :public AbstractApple {
21 public:
22     virtual void showName() {
23         cout << "美国苹果" << endl;
24     }
25 };
26
27 //日本苹果
28 class JapanApple :public AbstractApple {
29 public:
30     virtual void showName() {
31         cout << "日本苹果" << endl;
32     }
33 };
34
35 //香蕉的抽象
36 class AbstractBanana {
37

```

```

37 public:
38     virtual void showName() = 0;
39 };
40
41 //中国香蕉
42 class ChinaBanana :public AbstractBanana {
43 public:
44     virtual void showName() {
45         cout << "中国香蕉" << endl;
46     }
47 };
48
49 //美国香蕉
50 class USABanana :public AbstractBanana {
51 public:
52     virtual void showName() {
53         cout << "美国香蕉" << endl;
54     }
55 };
56
57 //日本香蕉
58 class JapanBanana :public AbstractBanana {
59 public:
60     virtual void showName() {
61         cout << "日本香蕉" << endl;
62     }
63 };
64
65 //鸭梨的抽象
66 class AbstractPear {
67 public:
68     virtual void showName() = 0;
69 };
70
71 //中国鸭梨
72 class ChinaPear :public AbstractPear {
73 public:
74     virtual void showName() {
75         cout << "中国鸭梨" << endl;
76     }
77 };
78
79 //美国鸭梨
80 class USAPear :public AbstractPear {
81 public:
82     virtual void showName() {
83         cout << "美国鸭梨" << endl;
84     }
85 };
86
87 //日本鸭梨
88 class JapanPear :public AbstractPear {
89 public:
90     virtual void showName() {
91         cout << "日本鸭梨" << endl;
92     }
93 };
94
95 //抽象工厂 针对产品族
96 class AbstractFactory {
97 public:
98     virtual AbstractApple* CreateApple() = 0;
99     virtual AbstractBanana* CreateBanana() = 0;
100     virtual AbstractPear* CreatePear() = 0;
101 };
102
103 //中国工厂
104 class ChinaFactory :public AbstractFactory {
105 public:
106     virtual AbstractApple* CreateApple() {
107         return new ChinaApple;
108     }

```

```

109     virtual AbstractBanana* CreateBanana() {
110         return new ChinaBanana;
111     }
112     virtual AbstractPear* CreatePear() {
113         return new ChinaPear;
114     }
115 };
116
117 //美国工厂
118 class USAFactory :public AbstractFactory {
119     virtual AbstractApple* CreateApple() {
120         return new USAApple;
121     }
122     virtual AbstractBanana* CreateBanana() {
123         return new USABanana;
124     }
125     virtual AbstractPear* CreatePear() {
126         return new USAPear;
127     }
128 };
129
130 //日本工厂
131 class JapanFactory :public AbstractFactory {
132     virtual AbstractApple* CreateApple() {
133         return new JapanApple;
134     }
135     virtual AbstractBanana* CreateBanana() {
136         return new JapanBanana;
137     }
138     virtual AbstractPear* CreatePear() {
139         return new JapanPear;
140     }
141 };
142
143 void test01() {
144     AbstractFactory* factory = NULL;
145     AbstractApple* apple = NULL;
146     AbstractBanana* Banana = NULL;
147     AbstractPear* Pear = NULL;
148
149     //中国工厂
150     factory = new ChinaFactory;
151     apple = factory->CreateApple();
152     Banana = factory->CreateBanana();
153     Pear = factory->CreatePear();
154
155     apple->showName();
156     Banana->showName();
157     Pear->showName();
158
159     delete Pear;
160     delete apple;
161     delete Banana;
162     delete factory;
163 }
164
165 int main()
166 {
167     test01();
168 }

```

总结:

- AbstractFactory 模式和 Factory模式的区别是初学（使用）设计模式时候的一个容易引起困惑的地方。实际上，AbstractFactory模式是为创建一组（有多类）相关或依赖的对象提供创建接口，而 Factory模式是为一类对象提供创建接口或延迟对象的创建到子类中实现。并且可以看到，AbstractFactory模式通常都是使用 Factory 模式实现。

Singleton 模式（单例模式）

Singleton 模式是设计模式中最为简单、最为常见、最容易实现，也是最应该熟悉和掌握的模式。Singleton 模式就是一个类只创建一个唯一的对象，即一次创建多次使用。

实现单例模式的步骤：

- 1、构造函数私有化
- 2、增加静态私有的当前类的指针变量
- 3、提供静态对外接口,可以让用户获得单例对象

单例分为懒汉式和饿汉式

懒汉式：解决了饿汉式内存浪费问题，但是线程不安全的，可以通过互斥量mutex.lock()和mutex.unlock()来解决

饿汉式：还没有使用该单例对象，该单例对象就已经被加载到内存了，在对象过多时会造成内存浪费

代码：

```
1  #include <iostream>
2  using namespace std;
3
4  class A {
5  public:
6      static A* getInstance() {
7          return a;
8      }
9
10 private :
11     A() {
12         a = new A;
13     }
14     static A* a;
15 };
16 A* A::a = NULL;
17
18 //懒汉式 对象的创建在第一次调用getInstance函数时创建
19 //懒汉式是线程不安全的
20 class SingletonLazy {
21 public:
22     static SingletonLazy* getInstance() {
23         if (pSingleton == NULL) {
24             pSingleton = new SingletonLazy;
25         }
26         return pSingleton;
27     }
28 private:
29     SingletonLazy() {}
30     static SingletonLazy* pSingleton;
31 };
32 //在类外面进行初始化
33 SingletonLazy* SingletonLazy::pSingleton=NULL;
34
35
36
37 //饿汉式 对象在程序执行时优先创建
38 //饿汉式是线程安全的
39 class SingletonHungry {
40 public:
41     static SingletonHungry* getInstance() {
42         return pSingleton;
43     }
44
45     static void freeSpace() {
46         if (pSingleton != NULL) {
47             delete pSingleton;
48         }
49     }
50 private:
51     SingletonHungry() {}
52     static SingletonHungry* pSingleton;
53 };
54
```



```

55 //以下语句将会在main函数运行前执行
56 SingletonHungry* SingletonHungry::pSingleton=new SingletonHungry;
57
58
59 void test01() {
60     SingletonLazy* p1 = SingletonLazy::getInstance();
61     SingletonLazy* p2 = SingletonLazy::getInstance();
62     if (p1 == p2) {
63         cout << "单例模式" << endl;
64     }
65     else {
66         cout << "不是单例模式" << endl;
67     }
68     SingletonHungry* p3 = SingletonHungry::getInstance();
69     SingletonHungry* p4 = SingletonHungry::getInstance();
70     if (p3 == p4) {
71         cout << "单例模式" << endl;
72     }
73     else {
74         cout << "不是单例模式" << endl;
75     }
76 }
77 int main()
78 {
79     test01();
80 }

```

总结:

- Singleton 不可以被实例化，因此将其构造函数声明private
- Singleton 模式经常和 Factory（AbstractFactory）模式在一起使用，因为系统中工厂对象一般来说只要一个

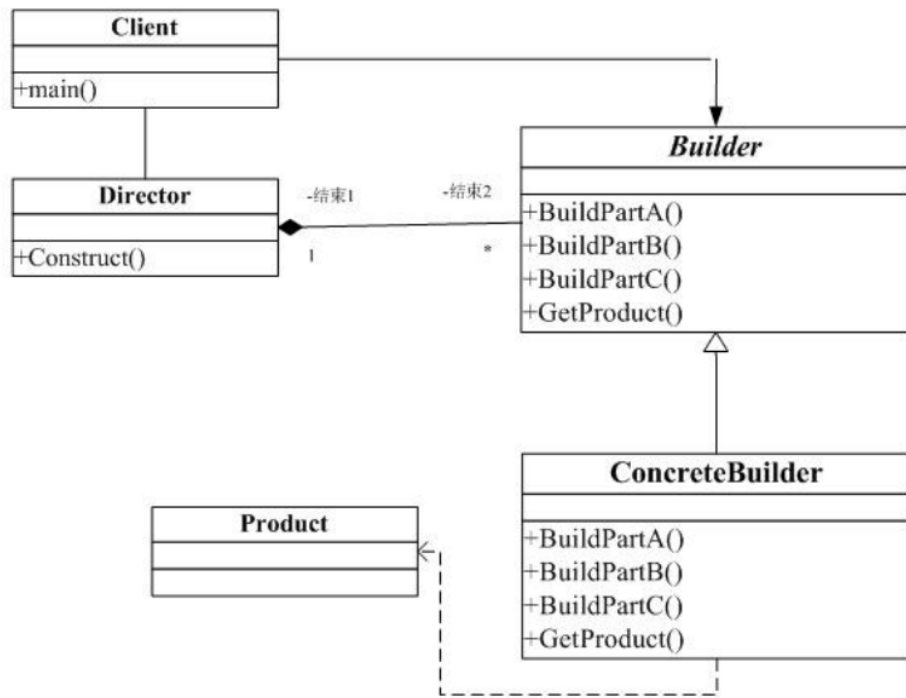
Builder 模式（建造者模式）

生活中有着很多的Builder的例子，个人觉得大学生活就是一个Builder模式的最好体验：要完成大学教育，一般将大学教育过程分成4个学期进行，因此没有学习可以看作是构建完整大学教育的一个部分构建过程，每个人经过这4年的（4个阶段）构建过程得到的最后的结果不一样，因为可能在四个阶段的构建中引入了很多的参数（每个人的机会和际遇不完全相同）

Builder 模式要解决的也正是这样的问题：当我们要创建的对象很复杂的时候（通常是由很多其他的对象组合而成），我们要复杂对象的创建过程和这个对象的表示（展示）分离开来，这样做的好处就是通过一步步的进行复杂对象的构建，由于在每一步的构造过程中可以引入参数，使得经过相同的步骤创建最后得到的对象的展示不一样。

举例:

Builder Pattern



CSDN @mrmrchohla

代码:

```

1  //Product.h
2  #ifndef _PRODUCT_H_
3  #define _PRODUCT_H_
4  class Product
5  {
6  public:
7      Product();
8      ~Product();
9      void ProducePart();
10 protected:
11 private:
12 };
13 class ProductPart
14 {
15 public:
16     ProductPart();
17     ~ProductPart();
18     ProductPart* BuildPart();
19 protected:
20 private:
21 };
22 #endif //~_PRODUCT_H_
23
24
25 //Component.cpp
26 #include "Component.h"
27 Component::Component()
28 {
29 }
30 Component::~Component()
31 {
32 }
33 void Component::Add(const Component&
34 com)
35 {
36 }
37 Component* Component::GetChild(int index)
38 {
39     return 0;
40 }

```

```

41 }
42 void Component::Remove(const Component&
43 com)
44 {
45 }
46
47
48 //Builder.h
49 #ifndef _BUILDER_H_
50 #define _BUILDER_H_
51 #include <string>
52 using namespace std;
53 class Product;
54 class Builder
55 {
56 public:
57     virtual ~Builder();
58     virtual void BuildPartA(const string&
59 buildPara) = 0;
60     virtual void BuildPartB(const string&
61 buildPara) = 0;
62     virtual void BuildPartC(const string&
63 buildPara) = 0;
64     virtual Product* GetProduct() = 0;
65 protected:
66     Builder();
67 private:
68 };
69 class ConcreteBuilder:public Builder
70 {
71 public:
72     ConcreteBuilder();
73     ~ConcreteBuilder();
74     void BuildPartA(const string&
75 buildPara);
76     void BuildPartB(const string& buildPara);
77     void BuildPartC(const string& buildPara);
78     Product* GetProduct();
79 protected:
80 private:
81 };
82 #endif //~_BUILDER_H_
83
84 //Builder.cpp
85 #include "Builder.h"
86 #include "Product.h"
87 #include <iostream>
88 using namespace std;
89 Builder::Builder()
90 {
91 }
92 Builder::~Builder()
93 {
94 }
95 ConcreteBuilder::ConcreteBuilder()
96 {
97 }
98 ConcreteBuilder::~~ConcreteBuilder()
99 {
100 }
101 void ConcreteBuilder::BuildPartA(const string& buildPara)
102 {
103     cout<<"Step1:Build PartA..."<<buildPara<<endl;
104 }
105 void ConcreteBuilder::BuildPartB(const string& buildPara)
106 {
107     cout<<"Step1:Build PartB..."<<buildPara<<endl;
108 }
109 void ConcreteBuilder::BuildPartC(const string& buildPara)
110 {
111     cout<<"Step1:Build PartC..."<<buildPara<<endl;

```

```

112 | }
113 | Product* ConcreteBuilder::GetProduct()
114 | {
115 |     BuildPartA("pre-defined");
116 |     BuildPartB("pre-defined");
117 |     BuildPartC("pre-defined");
118 |     return new Product();
119 | }
120 |
121 | //Director.h
122 | #ifndef _DIRECTOR_H_
123 | #define _DIRECTOR_H_
124 | class Builder;
125 | class Director
126 | {
127 | public:
128 |     Director(Builder* bld);
129 |     ~Director();
130 |     void Construct();
131 | protected:
132 | private:
133 |     Builder* _bld;
134 | };
135 | #endif //~_DIRECTOR_H_
136 |
137 |
138 | //Director.cpp
139 | #include "director.h"
140 | #include "Builder.h"
141 | Director::Director(Builder* bld)
142 | {
143 |     _bld = bld;
144 | }
145 | Director::~Director()
146 | {
147 |
148 | }
149 | void Director::Construct()
150 | {
151 |     _bld->BuildPartA("user-defined");
152 |     _bld->BuildPartB("user-defined");
153 |     _bld->BuildPartC("user-defined");
154 | }
155 |
156 | //main.cpp
157 | #include "Builder.h"
158 | #include "Product.h"
159 | #include "Director.h"
160 | #include <iostream>
161 | using namespace std;
162 | int main(int argc, char* argv[])
163 | {
164 |     Director* d = new Director(new
165 | ConcreteBuilder());
166 |     d->Construct();
167 |     return 0;
168 | }

```

总结:

- Builder 模式的关键是其中的 Director 对象并不直接返回对象，而是通过一步步（BuildPartA, BuildPartB, BuildPartC）来一步步进行对象的创建。
- Builder 模式的示例代码中，BuildPart 的参数是通过客户程序员传入的，这里使用“user-defined”代替，实际的可在 Construct 方法中传入这 3 个参数，这样就可以得到不同的细微差别的复杂对象了。

Builder 模式和 AbstractFactory 模式在功能上很相似，因为都是用来创建大的复杂的对象，它们的区别是 Builder 模式强调的是一步步创建对象，并通过相同的创建过程可以获得不同的结果对象，一般来说 Builder 模式中对象不是直接返回的。而在 AbstractFactory 模式中对象是直接返回的，AbstractFactory 模式强调的是为创建多个相互依赖的对象提供一个同一的接口。

Prototype 模式（原型模式）

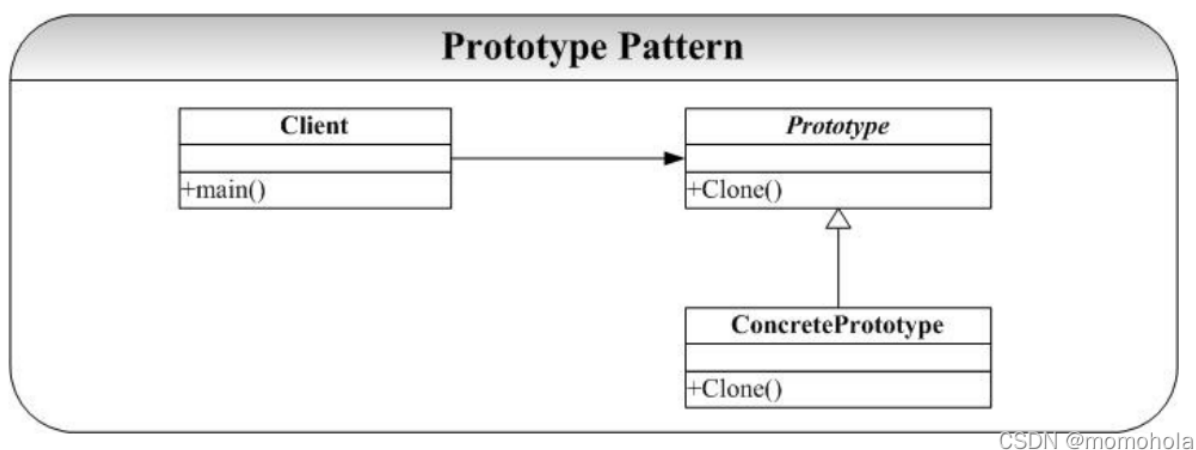
定义：Prototype 模式也正是提供了自我复制的功能，就是说新对象的创建可以通过已有对象进行创建。在 C++ 中拷贝构造函数（Copy Constructor）曾经是很对程序员的噩梦，拷贝又分为浅拷贝和深拷贝

浅拷贝：将一个对象复制后，基本数据类型的变量都会重新创建，而引用类型，指向的还是原对象所指向的。

深拷贝：将一个对象复制后，不论是基本数据类型还有引用类型，都是重新创建的。简单来说，就是深复制进行了完全彻底的复制，而浅复制不彻底。clone 明显是深复制，clone 出来的对象是不能去影响原型对象的

Prototype 模式提供了一个通过已存在对象进行新对象创建的接口（Clone），Clone 接口在 C++ 中我们将通过拷贝构造函数实现。

举例：



代码：

```
1 //Prototype.h
2 #ifndef _PROTOTYPE_H_
3 #define _PROTOTYPE_H_
4 class Prototype
5 {
6 public:
7     virtual ~Prototype();
8     virtual Prototype* Clone() const = 0;
9 protected:
10     Prototype();
11 private:
12 };
13 class ConcretePrototype:public Prototype
14 {
15 public:
16     ConcretePrototype();
17     ConcretePrototype(const
18 ConcretePrototype& cp);
19     ~ConcretePrototype();
20     Prototype* Clone() const;
21 protected:
22 private:
23 };
24 #endif //~_PROTOTYPE_H_
25
26
27 //Prototype.cpp
28 #include "Prototype.h"
29 #include <iostream>
30 using namespace std;
31 Prototype::Prototype()
32 {
33 }
34 Prototype::~~Prototype()
35 {
36 }
37 Prototype* Prototype::Clone() const
38
```

```

39 {
40     return 0;
41 }
42 ConcretePrototype::ConcretePrototype()
43 {
44 }
45 ConcretePrototype::~ConcretePrototype()
46 {
47 }
48 ConcretePrototype::ConcretePrototype(const
49 ConcretePrototype& cp)
50 {
51     cout<<"ConcretePrototype copy ..."<<endl;
52 }
53 Prototype* ConcretePrototype::Clone() const
54 {
55     return new ConcretePrototype(*this);
56 }
57
58
59 //main.cpp
60 #include "Prototype.h"
61 #include <iostream>
62 using namespace std;
63 int main(int argc, char* argv[])
64 {
65     Prototype* p = new ConcretePrototype();
66     Prototype* p1 = p->Clone();
67     return 0;
68 }

```

总结:

- Prototype 模式通过复制原型（Prototype）而获得新对象创建的功能，这里 Prototype本身就是“对象工厂”（因为能够生产对象），实际上 Prototype 模式和 Builder 模式、AbstractFactory模式都是通过一个类（对象实例）来专门负责对象的创建工作（工厂对象），它们之间的区别是：Builder模式重在复杂对象的一步步创建（并不直接返回对象），AbstractFactory 模式重在产生多个相互依赖类的对象，而 Prototype模式重在从自身复制自己创建新类。
- 原型模式的本质就是clone，可以解决构建复杂对象的资源消耗问题，能再某些场景中提升构建对象的效率；还有一个重要的用途就是保护性拷贝，可以通过返回一个拷贝对象的形式，实现只读的限制

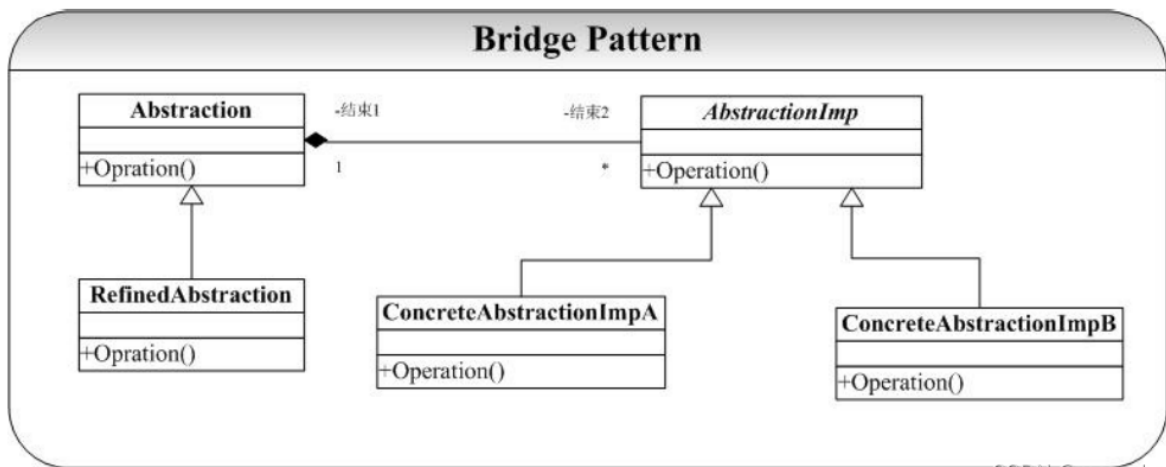
二、结构型模式

Bridge 模式（桥接模式）

假如遇到这样的问题：

- 1) 客户给了你一个需求，于是使用一个类来实现（A）；
- 2) 客户需求变化，有两个算法实现功能，于是改变设计，我们通过一个抽象的基类，再定义两个具体类实现两个不同的算法（A1和 A2）；
- 3) 客户又告诉我们说对于不同的操作系统，于是再抽象一个层次，作为一个抽象基类A0，在分别为每个操作系统派生具体类（A00和 A01，其中 A00 表示原来的类 A）实现不同操作系统上的客户需求，这样我们就有了一共 4 个类。
- 4) 可能用户的需求又有变化，比如说又有了一种新的算法.....
- 5) 我们陷入了一个需求变化的郁闷当中，也因此带来了类的迅速膨胀。Bridge 模式则正是解决了这类问题

示例:



CSDN@momohola

代码:

```
1 //Abstraction.h
2 #ifndef _ABSTRACTION_H_
3 #define _ABSTRACTION_H_
4 class AbstractionImp;
5 class Abstraction
6 {
7 public:
8     virtual ~Abstraction();
9     virtual void Operation() = 0;
10 protected:
11     Abstraction();
12 private:
13 };
14
15 class RefinedAbstraction:public Abstraction
16 {
17 public:
18     RefinedAbstraction(AbstractionImp* imp);
19     ~RefinedAbstraction();
20     void Operation();
21 protected:
22 private:
23     AbstractionImp* _imp;
24 };
25 #endif //~_ABSTRACTION_H_
26
27 //Abstraction.cpp
28 #include "Abstraction.h"
29 #include "AbstractionImp.h"
30 #include <iostream>
31 using namespace std;
32 Abstraction::Abstraction()
33 {
34 }
35 Abstraction::~~Abstraction()
36 {
37 }
38 RefinedAbstraction::RefinedAbstraction(AbstractionImp* imp)
39 {
40     _imp = imp;
41 }
42 RefinedAbstraction::~~RefinedAbstraction()
43 {
44 }
45 void RefinedAbstraction::Operation()
46 {
47     _imp->Operation();
48 }
49
50 //AbstractionImp.h
51
```

```

-
52 #ifndef _ABSTRACTIONIMP_H_
53 #define _ABSTRACTIONIMP_H_
54 class AbstractionImp
55 {
56 public:
57     virtual ~AbstractionImp();
58     virtual void Operation() = 0;
59 protected:
60     AbstractionImp();
61 private:
62 };
63 class ConcreteAbstractionImpA:public AbstractionImp
64 {
65 public:
66     ConcreteAbstractionImpA();
67     ~ConcreteAbstractionImpA();
68     virtual void Operation();
69 protected:
70 private:
71 };
72 class ConcreteAbstractionImpB:public AbstractionImp
73 {
74 public:
75     ConcreteAbstractionImpB();
76     ~ConcreteAbstractionImpB();
77     virtual void Operation();
78 protected:
79 private:
80 };
81 #endif //~_ABSTRACTIONIMP_H_
82
83 //AbstractionImp.cpp
84 #include "AbstractionImp.h"
85 #include <iostream>
86 using namespace std;
87 AbstractionImp::AbstractionImp()
88 {
89 }
90 AbstractionImp::~AbstractionImp()
91 {
92 }
93 void AbstractionImp::Operation()
94 {
95     cout<<"AbstractionImp....imp..."<<endl;
96 }
97 ConcreteAbstractionImpA::ConcreteAbstractionImpA()
98 {
99 }
100 ConcreteAbstractionImpA::~ConcreteAbstractionImpA()
101 {
102 }
103 void ConcreteAbstractionImpA::Operation()
104 {
105     cout<<"ConcreteAbstractionImpA...."<<endl;
106 }
107 ConcreteAbstractionImpB::ConcreteAbstractionImpB()
108 {
109 }
110 ConcreteAbstractionImpB::~ConcreteAbstractionImpB()
111 {
112 }
113 void ConcreteAbstractionImpB::Operation()
114 {
115     cout<<"ConcreteAbstractionImpB...."<<endl;
116 }
117
118 //main.cpp
119 #include "Abstraction.h"
120 #include "AbstractionImp.h"
121 #include <iostream>
122 using namespace std;

```



```

123 | int main(int argc, char* argv[])
124 | {
125 |     AbstractionImp* imp = new ConcreteAbstractionImpA();
126 |     Abstraction* abs = new RefinedAbstraction(imp);
127 |     abs->Operation();
128 |
129 |     return 0;
    | }

```

总结:

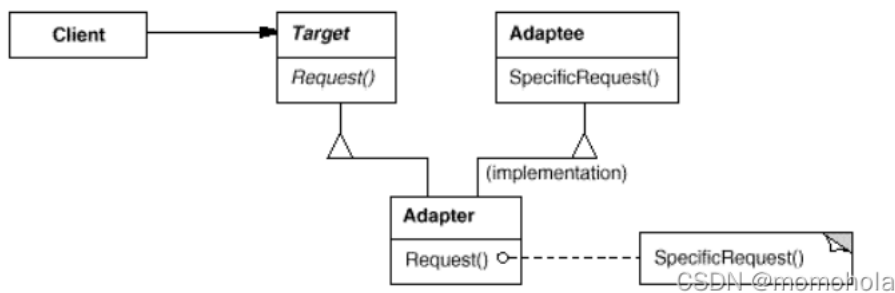
- 桥接模式实现了抽象化与实现化的脱耦。他们两个互相独立，不会影响到对方。
- 对于两个独立变化的维度，使用桥接模式再适合不过了。
- 分离抽象接口及其实现部分。提高了比继承更好的解决方案

Adapter 模式（适配器模式）

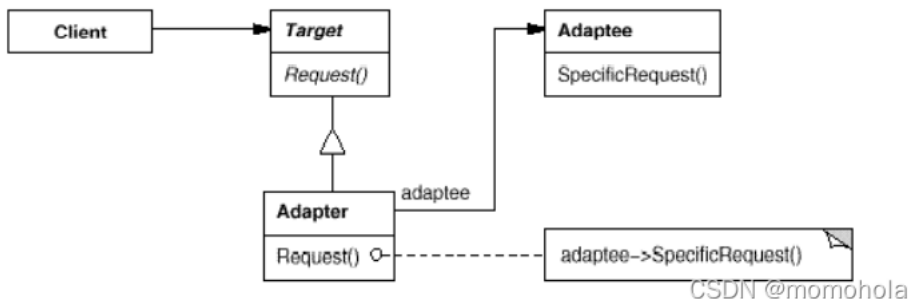
为了完成某项工作购买了一个第三方的库来加快开发。这就带来了一个问题：我们在应用程序中已经设计好了接口，与这个第三方提供的接口不一致，为了使得这些接口不兼容的类（不能在一起工作）可以在一起工作了，Adapter 模式提供了将一个类（第三方库）的接口转化希望的接口。

适配器模式分为类模式和对象模式。

1) 采用继承原有接口类的方式



2) 采用组合原有接口类的方式



在 Adapter 模式的结构图中可以看到，类模式的 Adapter 采用继承的方式复用 Adaptee 的接口，而在对象模式的 Adapter 中我们则采用组合的方式实现 Adaptee 的复用。

代码：（只演示采用组合的方式）

```

1 | //Adapter.h
2 | #ifndef _ADAPTER_H_
3 | #define _ADAPTER_H_
4 | class Target
5 | {
6 | public:
7 |     Target();
8 |     virtual ~Target();
9 |     virtual void Request();
10 | };
11 | class Adaptee
12 | {
13 | public:
14 |     Adaptee();
15 |     ~Adaptee();
16 |     void SpecificRequest();
17 |

```

```

18 };
19 class Adapter:public Target
20 {
21 public:
22     Adapter(Adaptee* ade);
23     ~Adapter();
24     void Request();
25 private:
26     Adaptee* _ade;
27 };
28 #endif //~_ADAPTER_H_
29
30
31 //Adapter.cpp
32 #include "Adapter.h"
33 #include <iostream>
34 Target::Target()
35 {
36 }
37 Target::~Target()
38 {
39 }
40 void Target::Request()
41 {
42     std::cout<<"Target::Request"<<std::endl;
43 }
44 Adaptee::Adaptee()
45 {
46 }
47 Adaptee::~Adaptee()
48 {
49 }
50 void Adaptee::SpecificRequest()
51 {
52     std::cout<<"Adaptee::SpecificRequest"<<std::endl;
53 }
54 Adapter::Adapter(Adaptee* ade)
55 {
56     this->_ade = ade;
57 }
58 Adapter::~Adapter()
59 {
60 }
61 void Adapter::Request()
62 {
63     _ade->SpecificRequest();
64 }
65
66 //main.cpp
67 #include "Adapter.h"
68 #include <iostream>
69 using namespace std;
70 int main(int argc,char* argv[])
71 {
72     Adaptee* ade = new Adaptee;
73     Target* adt = new Adapter(ade);
74     adt->Request();
75     return 0;
76 }

```

Decorator 模式（装饰器模式）

装饰器模式（Decorator Pattern）允许向一个现有的对象添加新的功能，同时又不改变其结构。

Component（被装饰对象的基类）

定义一个对象接口，可以给这些对象动态地添加职责。

ConcreteComponent（具体被装饰对象）

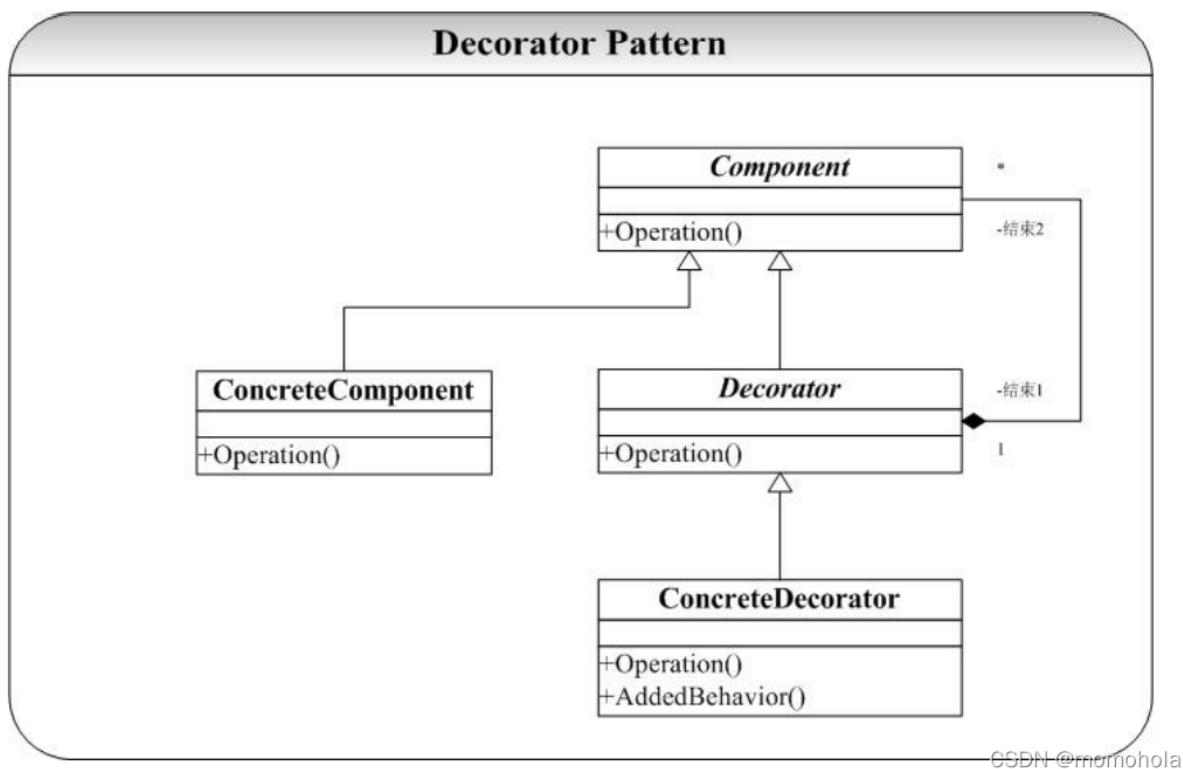
定义一个对象，可以给这个对象添加一些职责。

Decorator（装饰者抽象类）

维持一个指向Component实例的引用，并定义一个与Component接口一致的接口。

ConcreteDecorator（具体装饰者）

具体的装饰对象，给内部持有的具体被装饰对象，增加具体的职责。



```
1 //装饰器模式:对类的功能进行扩展(代替继承)。可以动态的给类增加功能
2 //我们需要为英雄增加额外的功能或者属性,但是又不能影响Hero类,这个时候就可以考虑装饰器模式
3 #include <iostream>
4 using namespace std;
5
6 //抽象的英雄
7 class AbstractHero
8 {
9 public:
10     virtual void showStatus() = 0;
11
12     int mHp;
13     int mMp;
14     int mAt;
15     int mDf;
16 };
17
18 class HeroA:public AbstractHero
19 {
20 public:
21     HeroA()
22     {
23         mHp = 0;
24         mMp = 0;
25         mAt = 0;
26         mDf = 0;
27     }
28
29     virtual void showStatus()
30     {
31         cout << "血量: " << mHp << endl;
32         cout << "魔法: " << mMp << endl;
33         cout << "攻击: " << mAt << endl;
34         cout << "防御: " << mDf << endl;
35     }
36 };
37
```

```

38 //英雄穿上某个装饰物, 那么也还是一个英雄
39 class AbstractEquipment: public AbstractHero
40 {
41 public:
42     AbstractEquipment(AbstractHero* hero)
43     {
44         this->pHero = hero;
45     }
46
47     virtual void showStatus()
48     {
49     }
50
51     AbstractHero* pHero;
52 };
53
54 //给英雄穿上狂徒铠甲
55 class KuangtuEquipment:public AbstractEquipment
56 {
57 public:
58     KuangtuEquipment(AbstractHero* hero):AbstractEquipment(hero){}
59
60     //增加额外的功能
61     void AddKuangtu()
62     {
63         cout << "英雄穿了狂徒之后" << endl;
64         this->mHp = this->pHero->mHp;
65         this->mMp = this->pHero->mMp;
66         this->mAt = this->pHero->mAt;
67         this->mDf = this->pHero->mDf+30;
68     }
69
70
71     virtual void showStatus()
72     {
73         AddKuangtu();
74         cout << "血量: " << mHp << endl;
75         cout << "魔法: " << mMp << endl;
76         cout << "攻击: " << mAt << endl;
77         cout << "防御: " << mDf << endl;
78     }
79 };
80
81 //无尽之刃
82 class Wujinzhireng:public AbstractEquipment
83 {
84 public:
85     Wujinzhireng(AbstractHero* hero) :AbstractEquipment(hero) {}
86
87     //增加额外的功能
88     void AddKuangtu()
89     {
90         cout << "英雄穿了无尽之刃后" << endl;
91         this->mHp = this->pHero->mHp;
92         this->mMp = this->pHero->mMp;
93         this->mAt = this->pHero->mAt+80;
94         this->mDf = this->pHero->mDf;
95     }
96
97
98     virtual void showStatus()
99     {
100         AddKuangtu();
101         cout << "血量: " << mHp << endl;
102         cout << "魔法: " << mMp << endl;
103         cout << "攻击: " << mAt << endl;
104         cout << "防御: " << mDf << endl;
105     }
106 };
107
108 void test01()
109

```

```

109 {
110     AbstractHero* hero = new HeroA;
111     hero->showStatus();
112
113     cout << "-----" << endl;
114
115     //给英雄穿上狂徒
116     hero = new KuangtuEquipment(hero);
117     hero->showStatus();
118     hero->mAt = 33;
119     //给英雄装备武器
120     hero = new Wujinzhireng(hero);
121
122     hero->showStatus();
123 }
124
125 int main()
126 {
127     test01();
128 }

```

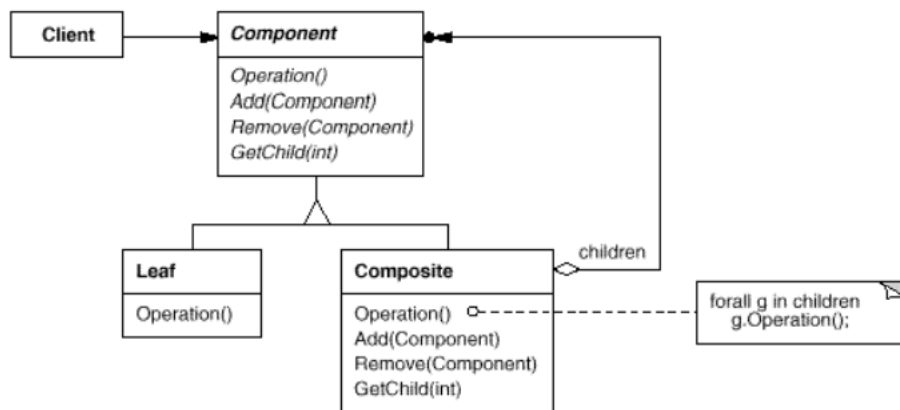
总结:

Decorator 模式除了采用组合的方式取得了比采用继承方式更好的效果, Decorator 模式还给设计带来一种“即用即付”的方式来添加职责。在 OO 设计和分析经常有这样一种情况: 为了多态, 通过父类指针指向其具体子类, 但是这就带来另外一个问题, 这样处于高层的父类就承载了太多的特征(方法), 并且继承自这个父类的所有子类都不可避免继承了父类的这些接口, 但是可能这并不是这个具体子类所需要的。这样父类就太过臃肿了, 那么当需要添加一个操作的时候就可以通过Decorator 模式来解决。

Composite 模式 (组合实体模式)

将对象组合成树形结构以表示“部分-整体”的层次结构。Composite 使得用户对单个对象和组合对象的使用具有一致性。

举例:



CSDN @momohola

代码:

```

1 //Composite.h
2 #ifndef COMPOSITE_H
3 #define COMPOSITE_H
4 #include <list>
5 // 组合中的抽象基类
6 class Component
7 {
8 public:
9     Component(){}
10    virtual ~Component(){}
11    // 纯虚函数, 只提供接口, 没有默认的实现
12    virtual void Operation() = 0;
13    // 虚函数, 提供接口, 有默认的实现就是什么都不做
14    virtual void Add(Component* pChild);
15    virtual void Remove(Component* pChild);
16    virtual Component* GetChild(int nIndex);
17 };
18 // 派生自 Component, 是其中的叶子组件的基类
19 class Leaf
20

```

```

21 : public Component
22 {
23 public:
24     Leaf(){}
25     virtual ~Leaf(){}
26     virtual void Operation();
27 };
28 // 派生自 Component, 是其中的含有子件的组件的基类
29 class Composite
30 : public Component
31 {
32 public:
33     Composite(){}
34     virtual ~Composite();
35     virtual void Operation();
36     virtual void Add(Component* pChild);
37     virtual void Remove(Component* pChild);
38     virtual Component* GetChild(int nIndex);
39 private:
40     // 采用 list 容器去保存子组件
41     std::list<Component*> m_ListOfComponent;
42 };
43 #endif
44
45 //Composite.cpp
46 #include "Composite.h"
47 #include <iostream>
48 #include <algorithm>
49 /*-----*/
50 Component 成员函数的实现
51 -----*/
52 void Component::Add(Component* pChild)
53 {
54 }
55 void Component::Remove(Component* pChild)
56 {
57 }
58 Component* Component::GetChild(int nIndex)
59 {
60     return NULL;
61 }
62 /*-----*/
63 Leaf 成员函数的实现
64 -----*/
65 void Leaf::Operation()
66 {
67     std::cout << "Operation by leaf\n";
68 }
69 /*-----*/
70 Composite 成员函数的实现
71 -----*/
72 Composite::~Composite()
73 {
74     std::list<Component*>::iterator iter1, iter2, temp;
75     PDF created with pdfFactory trial version www.pdffactory.com
76     for (iter1 = m_ListOfComponent.begin(), iter2 = m_ListOfComponent.end();
77          iter1 != iter2;
78          )
79     {
80         temp = iter1;
81         ++iter1;
82         delete (*temp);
83     }
84 }
85 void Composite::Add(Component* pChild)
86 {
87     m_ListOfComponent.push_back(pChild);
88 }
89 void Composite::Remove(Component* pChild)
90 {
91     std::list<Component*>::iterator iter;
92 }

```

```

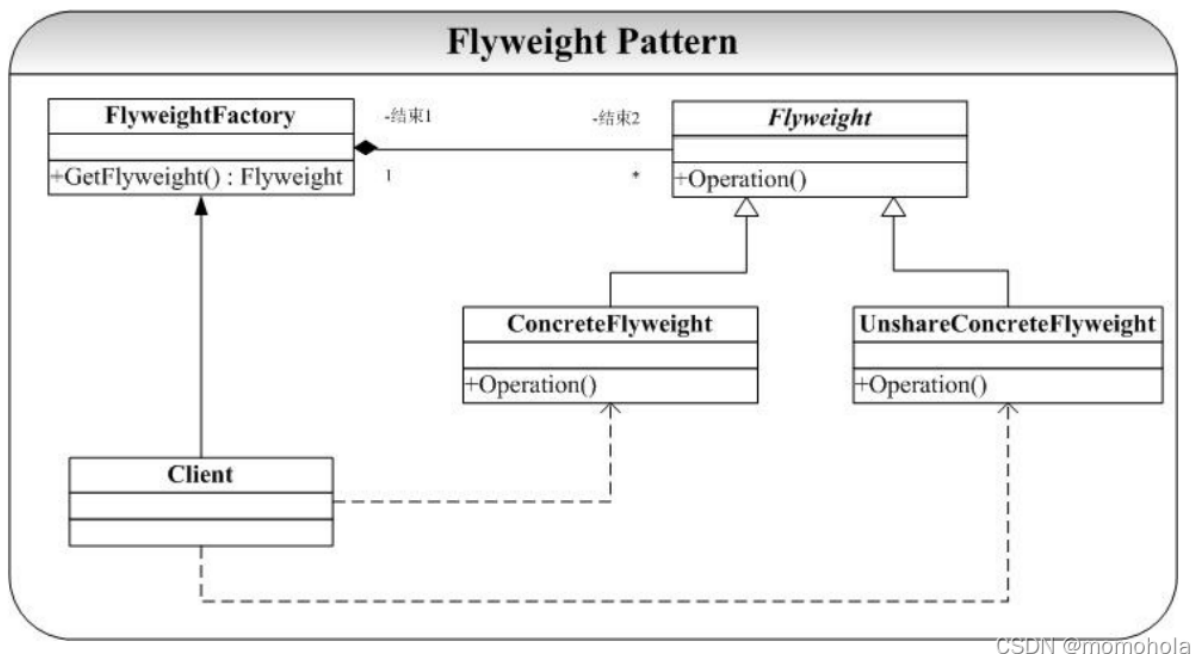
92 | iter = find(m_ListOfComponent.begin(), m_ListOfComponent.end(), pChild);
93 | if (m_ListOfComponent.end() != iter)
94 | {
95 |     m_ListOfComponent.erase(iter);
96 | }
97 | }
98 | Component* Composite::GetChild(int nIndex)
99 | {
100 |     if (nIndex <= 0 || nIndex > m_ListOfComponent.size())
101 |         return NULL;
102 |     std::list<Component*>::iterator iter1, iter2;
103 |     int i;
104 |     for (i = 1, iter1 = m_ListOfComponent.begin(), iter2 = m_ListOfComponent.
105 | end();
106 | iter1 != iter2;
107 | ++iter1, ++i)
108 | {
109 |     if (i == nIndex)
110 |         break;
111 | }
112 | PDF created with pdfFactory trial version www.pdffactory.com
113 | return *iter1;
114 | }
115 | void Composite::Operation()
116 | {
117 |     std::cout << "Operation by Composite\n";
118 |     std::list<Component*>::iterator iter1, iter2;
119 |     for (iter1 = m_ListOfComponent.begin(), iter2 = m_ListOfComponent.end();
120 | iter1 != iter2;
121 | ++iter1)
122 | {
123 |     (*iter1)->Operation();
124 | }
125 | }
126 |
127 | #include "Composite.h"
128 | #include <stdlib.h>
129 | int main()
130 | {
131 |     Leaf *pLeaf1 = new Leaf();
132 |     Leaf *pLeaf2 = new Leaf();
133 |     Composite* pComposite = new Composite;
134 |     pComposite->Add(pLeaf1);
135 |     pComposite->Add(pLeaf2);
136 |     pComposite->Operation();
137 |     pComposite->GetChild(2)->Operation();
    delete pComposite;

```

Flyweight 模式（享元模式）

如果一个应用程序使用了太多的对象，就会造成很大的存储开销。特别是对于大量轻量级（细粒度）的对象，比如在文档编辑器的设计过程中，我们如果为没有字母创建一个对象的话，系统可能会因为大量的对象而造成存储开销的浪费。例如一个字母“a”在文档中出现了100000次，而实际上我们可以让这一万个字母“a”共享一个对象，当然因为在不同的位置可能字母“a”有不同的显示效果

（例如字体和大小等设置不同），在这种情况下我们可以为将对象的状态分为“外部状态”和“内部状态”，将可以被共享（不会变化）的状态作为内部状态存储在对象中，而外部对象（例如上面提到的字体、大小等）我们可以在适当的时候将外部对象最为参数传递给对象（例如在显示的时候，将字体、大小等信息传递给对象）。



CSDN @momohola

Flyweight 模式中有一个类似 Factory 模式的对象构造工厂 FlyweightFactory，当客户程序员（Client）需要一个对象时候就会向 FlyweightFactory 发出请求对象的消息 GetFlyweight（）消息，FlyweightFactory 拥有一个管理、存储对象的“仓库”（或者叫对象池，vector 实现），GetFlyweight（）消息会遍历对象池中的对象，如果已经存在则直接返回给 Client，否则创建一个新的对象返回给 Client。

代码：

```

1 //Flyweight.h
2 #ifndef FLYWEIGHT_H
3 #define FLYWEIGHT_H
4 #include <string>
5 #include <list>
6 typedef std::string STATE;
7 class Flyweight
8 {
9 public:
10 virtual ~Flyweight(){}
11 STATE GetIntrinsicState();
12 virtual void Operation(STATE& ExtrinsicState) = 0;
13 protected:
14 Flyweight(const STATE& state)
15 :m_State(state)
16 {
17 }
18 private:
19 STATE m_State;
20 };
21 class FlyweightFactory
22 {
23 public:
24 FlyweightFactory(){}
25 ~FlyweightFactory();
26 Flyweight* GetFlyweight(const STATE& key);
27 private:
28 std::list<Flyweight*> m_listFlyweight;
29 };
30 class ConcreateFlyweight
31 : public Flyweight
32 {
33 public:
34 ConcreateFlyweight(const STATE& state)
35 : Flyweight(state)
36 {
37 }
38 virtual ~ConcreateFlyweight(){}
39 virtual void Operation(STATE& ExtrinsicState);
40 };
41 #endif
  
```



```

42 |
43 | //Flyweight.cpp
44 | #include "FlyWeight.h"
45 | #include <iostream>
46 | inline STATE Flyweight::GetIntrinsicState()
47 | {
48 |     return m_State;
49 | }
50 | FlyweightFactory::~FlyweightFactory()
51 | {
52 |     std::list<Flyweight*>::iterator iter1, iter2, temp;
53 |     for (iter1 = m_listFlyweight.begin(), iter2 = m_listFlyweight.end();
54 |          iter1 != iter2;
55 |          )
56 |     {
57 |         temp = iter1;
58 |         ++iter1;
59 |         delete (*temp);
60 |     }
61 |     m_listFlyweight.clear();
62 | }
63 | Flyweight* FlyweightFactory::GetFlyweight(const STATE& key)
64 | {
65 |     std::list<Flyweight*>::iterator iter1, iter2;
66 |     for (iter1 = m_listFlyweight.begin(), iter2 = m_listFlyweight.end();
67 |          iter1 != iter2;
68 |          ++iter1)
69 |     {
70 |         if ((*iter1)->GetIntrinsicState() == key)
71 |         {
72 |             std::cout << "The Flyweight:" << key << " already exists"<< std::endl;
73 |             return (*iter1);
74 |         }
75 |     }
76 |     std::cout << "Creating a new Flyweight:" << key << std::endl;
77 |     Flyweight* flyweight = new ConcreateFlyweight(key);
78 |     m_listFlyweight.push_back(flyweight);
79 | }
80 | void ConcreateFlyweight::Operation(STATE& ExtrinsicState)
81 | {
82 | }
83 |
84 | #include "FlyWeight.h"
85 | int main()
86 | {
87 |     FlyweightFactory flyweightfactory;
88 |     flyweightfactory.GetFlyweight("hello");
89 |     flyweightfactory.GetFlyweight("world");
90 |     flyweightfactory.GetFlyweight("hello");
91 |     system("pause");
92 |     return 0;
93 | }

```

****适用性:****一个应用程序使用了大量对象；完全由于使用大量的对象造成很大的存储开销；对象的大多数状态都可变为外部状态；如果删除对象的外部状态，那么可以使用相对较少的共享对象取代外部对象。

Facade 模式（外观模式）

外观模式是一个很重要、平常也用得很多的一个设计模式

实际上在软件系统开发中也经常会遇到这样的情况，可能你实现了一些接口（模块），而这些接口（模块）都分布在几个类中（比如 A 和 B、C、D）：A 中实现了一些接口，B 中实现一些接口（或者 A 代表一个独立模块，B、C、D 代表另一些独立模块）。然后你的客户程序员（使用你设计的开发人员）只有很少的要知道你的不同接口到底是在那个类中实现的，绝大多数只是想简单的组合你的 A - D 的类的接口，他并不想知道这些接口在哪里实现的。

下面举一个KTV的例子。假如KTV里面有电视机、电灯、音响、DVD、游戏机这些设备。平常KTV包厢里面没人时电灯都是打开的。电视机、音响、游戏机、DVD都是关闭的。当KTV里面有了那么电灯关闭，其他东西打开。如果要一个一个开和关特别麻烦。这时候就可以使用外观模式，定义一个总开关。

代码:

```
1 //外观模式
2 #include <iostream>
3 using namespace std;
4
5 //电视机类
6 class Television {
7 public:
8     void on() {
9         cout << "电视机打开" << endl;
10    }
11    void off() {
12        cout << "电视机关闭" << endl;
13    }
14 };
15
16 //灯类
17 class Light {
18 public:
19     void on() {
20         cout << "灯打开" << endl;
21    }
22    void off() {
23        cout << "灯关闭" << endl;
24    }
25 };
26
27 //音响
28 class Audio {
29 public:
30     void on() {
31         cout << "音响打开" << endl;
32    }
33     void off() {
34         cout << "音响关闭" << endl;
35    }
36 };
37
38 //麦克风
39 class Microphone {
40 public:
41     void on() {
42         cout << "麦克风打开" << endl;
43    }
44     void off() {
45         cout << "麦克风关闭" << endl;
46    }
47 };
48
49 //DVD
50 class DVDplayer {
51 public:
52     void on() {
53         cout << "DVD打开" << endl;
54    }
55     void off() {
56         cout << "DVD关闭" << endl;
57    }
58 };
59
60 //游戏机
61 class GameMachine {
62 public:
63     void on() {
64         cout << "游戏机打开" << endl;
65    }
66     void off() {
67         cout << "游戏机关闭" << endl;
68    }
69 };
```

```

70
71 //外观模式
72 class KTVMode {
73 public:
74     KTVMode() {
75         pTV = new Television;
76         pLight = new Light;
77         pAudio = new Audio;
78         pMicrophone = new Microphone;
79         pDvd = new DVDplayer;
80     }
81
82     void onKTV() {
83         pTV->on();
84         pLight->off();
85         pAudio->on();
86         pMicrophone->on();
87         pDvd->on();
88     }
89
90     void offKTV() {
91         pTV->off();
92         pLight->off();
93         pAudio->off();
94         pMicrophone->off();
95         pDvd->off();
96     }
97
98     ~KTVMode() {
99         delete pTV;
100         delete pLight;
101         delete pAudio;
102         delete pMicrophone;
103         delete pDvd;
104     }
105 private:
106     Television* pTV;
107     Light* pLight;
108     Audio* pAudio;
109     Microphone* pMicrophone;
110     DVDplayer* pDvd;
111 };
112
113 void test01() {
114     KTVMode* ktv = new KTVMode;
115     //KTV包厢来人了
116     ktv->onKTV();
117 }
118
119 int main()
120 {
121     test01();
122 }

```

Proxy 模式（代理模式）

代理模式（Proxy Pattern）是指为其他对象提供一种代理，以控制对这个对象的访问。代理对象在客户端和目标对象之间起到中介作用。

在生活中，我们经常见到这样的场景，如：租房中介、售票黄牛、婚介、经纪人、快递、事务代理、非侵入式日志监听等，这些都是代理模式的实际体现

注意事项：

- 1、和适配器模式的区别：适配器模式主要改变所考虑对象的接口，而代理模式不能改变所代理类的接口。
- 2、和装饰器模式的区别：装饰器模式为了增强功能，而代理模式是为了加以控制。

举例：

现在有一个操作系统，只需要调用run（）就可以启动操作系统，但是进入操作系统之前必须要进行账户名和密码的认证。认证成功后这个代理才会让你进入操作系统，其中认证的这个过程就是一个代理。

代码:

```
1 //代理模式: 提供一种代理来控制其他对象的访问
2 #include <iostream>
3 using namespace std;
4
5 class AbstractCommonInterface {
6 public:
7     virtual void run() = 0;
8 };
9 //下面是操作系统类
10 class MySystem :public AbstractCommonInterface{
11 public:
12     virtual void run() {
13         cout << "系统启动" << endl;
14     }
15 };
16
17 //代理: 启动系统必须要权限验证, 不是所以的人都可以来启动我的系统, 必须要提供用户名和密码
18 class MySystemProxy :public AbstractCommonInterface {
19 public:
20     MySystemProxy(string userName, string password) {
21         this->mUserName = userName;
22         this->mPassword = password;
23         pMySystem = new MySystem;
24     }
25
26     bool checkUserNameAndPassword() {
27         if (mUserName == "admin" && mPassword == "admin") {
28             return true;
29         }
30         return false;
31     }
32     virtual void run() {
33         if (checkUserNameAndPassword() == true) {
34             cout << "启动成功" << endl;
35             this->pMySystem->run();
36         }
37         else {
38             cout << "用户名或者密码错误, 权限不足" << endl;
39         }
40     }
41     ~MySystemProxy() {
42         if (pMySystem != NULL) {
43             delete pMySystem;
44         }
45     }
46 private:
47     string mUserName;
48     string mPassword;
49     MySystem* pMySystem;
50 };
51 void test01() {
52     MySystemProxy* proxy = new MySystemProxy("admin", "admin");
53     proxy->run();
54 }
55
56 int main()
57 {
58     test01();
59 }
```

总结:

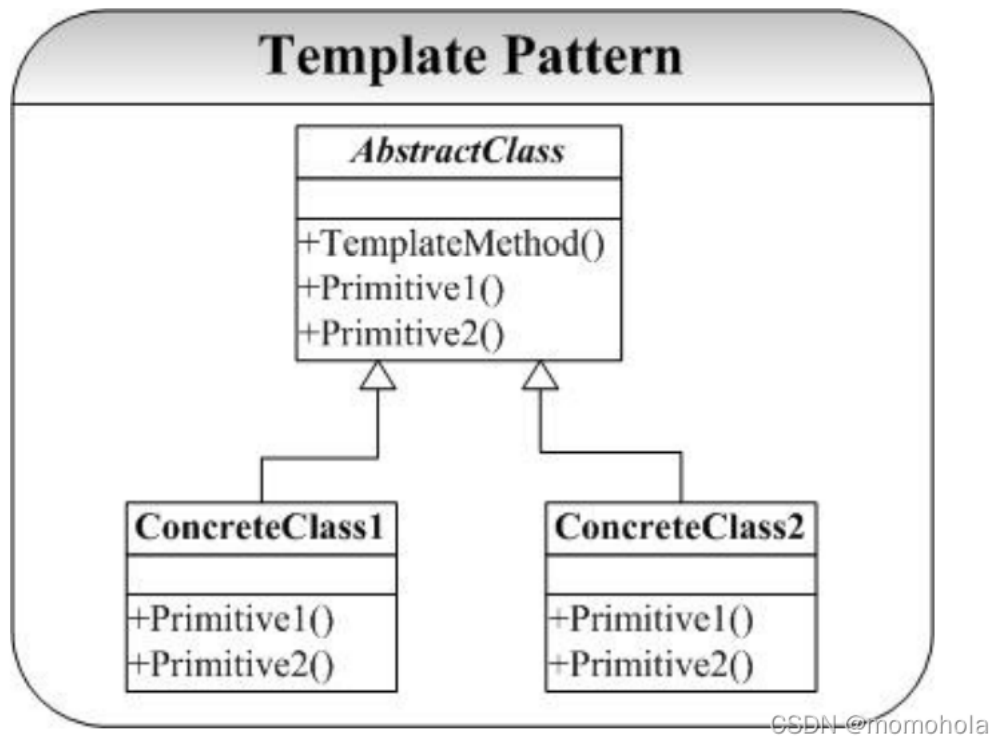
- 1、和适配器模式的区别: 适配器模式主要改变所考虑对象的接口, 而代理模式不能改变所代理类的接口。
- 2、和装饰器模式的区别: 装饰器模式为了增强功能, 而代理模式是为了加以控制

三、行为模式

Template 模式（模板模式）

在面向对象系统的分析与设计过程中经常会遇到这样一种情况：对于某一个业务逻辑（算法实现）在不同的对象中有不同的细节实现，但是逻辑（算法）的框架（或通用的应用算法）是相同的。Template 提供了这种情况的一个实现框架。Template 模式是采用继承的方式实现这一点：将逻辑（算法）框架放在抽象基类中，并定义好细节的接口，子类中实现细节。

举例：



假如我们要做饮料，那么我们首先会定义一个做饮料的模板，即煮水->冲泡->导入杯中->加辅助材料，具体煮什么水、冲泡什么东西放到子类中实现。然后定义一个模板方法，当我们要做饮料时就调用这个方法即可。

代码：

```
1 //模板方法模式: 在父类中定义一个方法的抽象, 由它的子类来实现细节的处理, 在子类实现详细的处理算法时并不会改变算法中的执行次
2 #include <iostream>
3 using namespace std;
4
5 //做饮料模板
6 class DrinkTemplate {
7 public:
8     //煮水
9     virtual void BoildWater() = 0;
10    //冲泡
11    virtual void Brew() = 0;
12    //倒入杯中
13    virtual void PourInCup() = 0;
14    //加辅助材料
15    virtual void AddSomething() = 0;
16
17    //模板方法
18    void Make() {
19        BoildWater();
20        Brew();
21        PourInCup();
22        AddSomething();
23    }
24 };
25
26 //做咖啡: 实现做饮料模板
27 class Coffee :public DrinkTemplate {
28     virtual void BoildWater() {
29         cout << "煮山泉水" << endl;
30     }
31     virtual void Brew() {
32         cout << "冲泡咖啡" << endl;
33     }
34 }
```

```

34     virtual void PourInCup() {
35         cout << "咖啡倒入杯中" << endl;
36     }
37     virtual void AddSomething() {
38         cout << "加糖, 加牛奶" << endl;
39     }
40 };
41
42 //做茶: 实现做饮料模板
43 class Tea :public DrinkTemplate {
44     virtual void BoildWater() {
45         cout << "煮自来水" << endl;
46     }
47     virtual void Brew() {
48         cout << "冲泡铁观音" << endl;
49     }
50     virtual void PourInCup() {
51         cout << "茶水倒入杯中" << endl;
52     }
53     virtual void AddSomething() {
54         cout << "加糖,加大蒜" << endl;
55     }
56 };
57
58 void test01() {
59     Tea* tea = new Tea;
60     tea->Make();
61
62     Coffee* coffee = new Coffee;
63     coffee->Make();
64 }
65
66 int main()
67 {
68     test01();
69 }
70

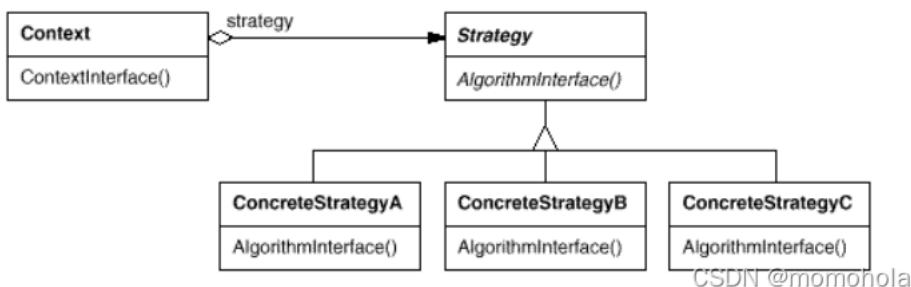
```

总结:

- 可以看到Template模式获得一种反向控制结构效果, 这也是面向对象系统的分析和设计中一个原则, 依赖倒置原则。其含义就是父类调用子类的操作 (高层模块调用低层模块的操作), 低层模块实现高层模块声明的接口。这样控制权在父类 (高层模块), 低层模块反而要依赖高层模块。
- 但是也有局限性, 因为模板模式采用继承这种强制约束关系, 导致复用性不高。假如我接下来要做冷饮, 那么Make函数中的煮水操作其实可以省去 直接冲泡即可, 但是我们就无法再次使用这个模板了, 这样就导致了不能复用子类的实现步骤。

Strategy 模式 (策略模式)

Strategy 模式和 Template 模式要解决的问题是相同 (类似) 的, 都是为了给业务逻辑 (算法) 具体实现和抽象接口之间的解耦。



简而言之,Strategy 模式是对算法的封装。处理一个问题的时候可能有多种算法,这些算法的接口(输入参数,输出参数等)都是一致的,那么可以考虑采用Strategy 模式对这些算法进行封装,在基类中定义一个函数接口就可以了。

举例:

假如现在有个英雄需要使用武器对付敌人, 武器有两种匕首和AK, 那么这么选择使用哪吧武器其实就是一种策略了那么就可以将策略模式分为三部分:

- Strategy 策略基类（抽象武器）
- ConcreteStrategy 具体策略（使用匕首或AK）
- Context 具体使用策略的对象（英雄）

代码:

```
1  //策略模式
2  #include <iostream>
3  using namespace std;
4
5  //抽象武器 策略基类 (抽象的策略)
6  class WeaponStrategy {
7  public:
8      virtual void UseWeapon() = 0;
9  };
10
11 //具体的策略使用匕首作为武器
12 class Knife :public WeaponStrategy {
13 public:
14     virtual void UseWeapon() {
15         cout << "使用匕首" << endl;
16     }
17 };
18
19 //具体的策略使用AK47作为武器
20 class AK47 :public WeaponStrategy {
21 public:
22     virtual void UseWeapon() {
23         cout << "使用AK47" << endl;
24     }
25 };
26
27 //具体使用策略的角色
28 class Character {
29 public:
30     WeaponStrategy* pWeapon;
31     void setWeapon(WeaponStrategy* pWeapon) {
32         this->pWeapon = pWeapon;
33     }
34
35     void ThrowWeapon() {
36         this->pWeapon->UseWeapon();
37     }
38 };
39
40 void test01() {
41     Character* character = new Character;
42     WeaponStrategy* knife = new Knife;
43     WeaponStrategy* ak47 = new AK47;
44
45     //用匕首当作武器
46     character->setWeapon(knife);
47     character->ThrowWeapon();
48
49     character->setWeapon(ak47);
50     character->ThrowWeapon();
51
52     delete ak47;
53     delete knife;
54     delete character;
55 }
56
57 int main()
58 {
59     test01();
60 }
```

总结:

可以看到Character 类是通过组合WeaponStrategy 的形式调用武器使用的，这就比继承要好得多，因为在面向对象的设计中的有一条很重要的原则就是：**优先使用对象组合，而非类继承。**

优点:

- 算法可以自由切换。
- 避免使用多重条件判断。
- 扩展性良好。

缺点:

- 策略类会增多。
- 所有策略类都需要对外暴露

State 模式（状态模式）

每个人、事物在不同的状态下会有不同表现（动作），而一个状态又会在不同的表现下转移到下一个不同的状态（State）。

主要解决:

1. 当状态数目不是很多的时候，Switch/Case 可能可以搞定。但是当状态数目很多的时候（实际系统中也正是如此），维护一大组的Switch/Case 语句将是一件异常困难并且容易出错的事情。
2. 状态逻辑和动作实现没有分离。在很多的系统实现中，动作的实现代码直接写在状态的逻辑当中。这带来的后果就是系统的扩展性和维护得不到保证。

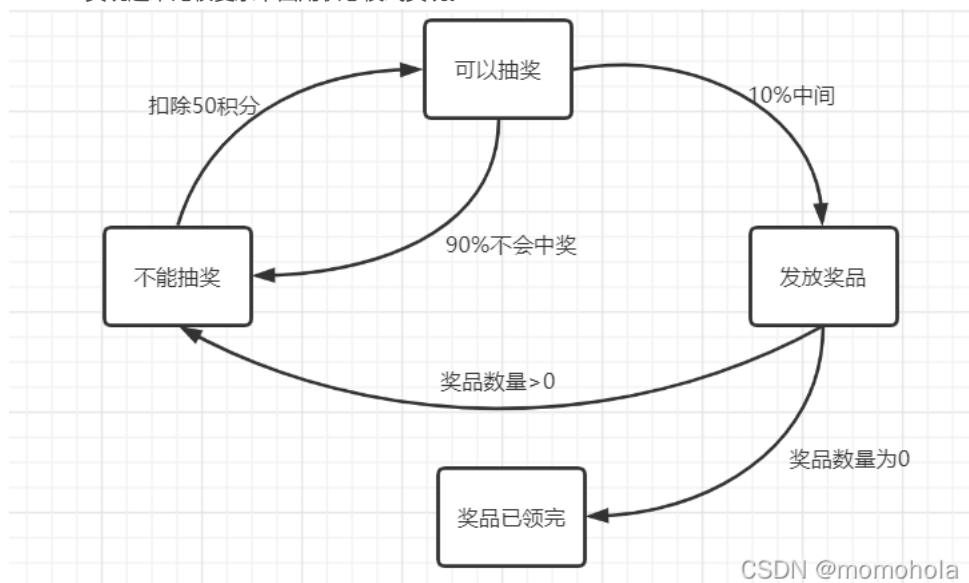
举例:

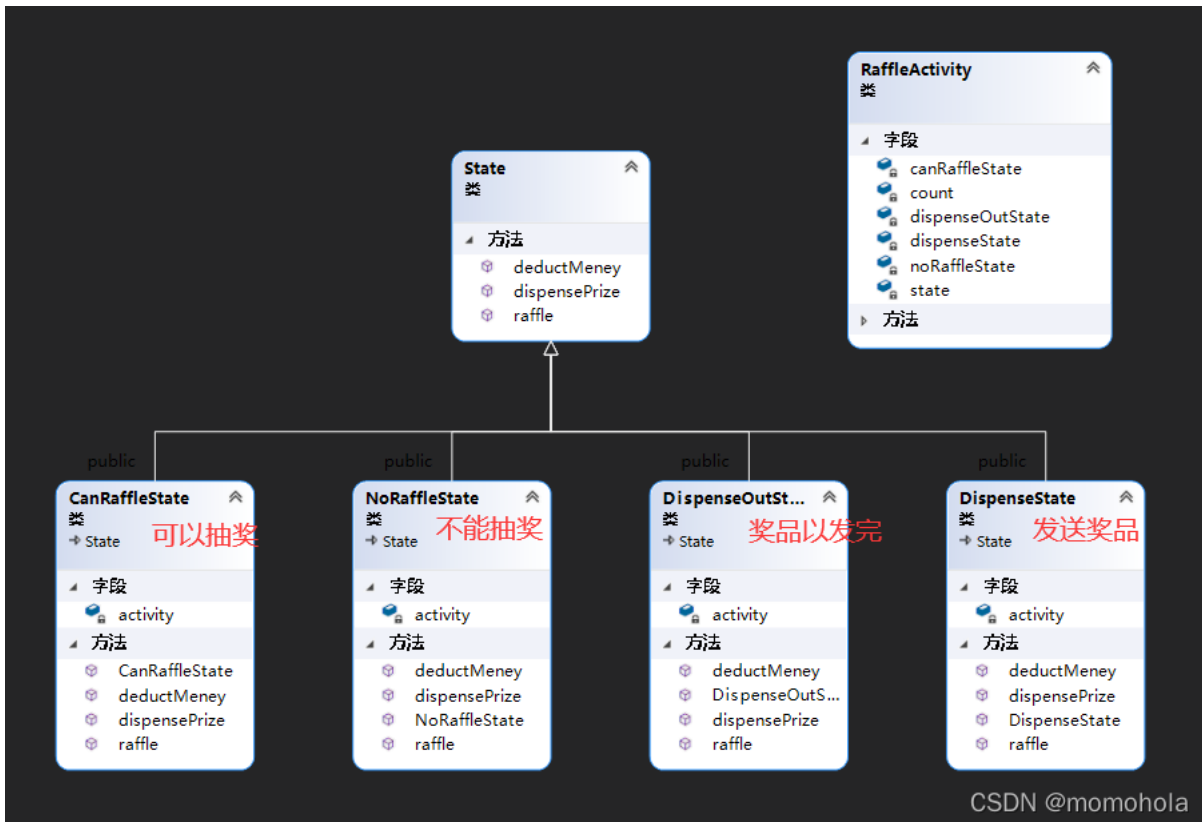
以抽奖活动为例。

有四个状态:

- 【抽奖状态】
- 【可以抽奖】
- 【发放奖品】
- 【奖品已领完】

抽奖活动的初始状态为【不能抽奖】状态，在扣除50积分后进入【可以抽奖】状态，【可以抽奖】状态有90%的几率不中奖，不中奖就会回到【抽奖状态】，10%的几率中奖，中奖后进入【发放奖品】状态，当奖品数量为0是进入【奖品已领完】状态。如果用switch/if实现起来比较复杂下面用状态模式实现。





CSDN @momohola

```

1 //State.h
2 #pragma once
3 class State
4 {
5 public:
6     //扣除50积分
7     virtual void deductMeney() = 0;
8
9     //是否中奖
10    virtual bool raffle() = 0;
11
12    //发放奖品
13    virtual void dispensePrize() = 0;
14 };
15
16 //CanRaffleState.h
17 #pragma once
18 #include <iostream>
19 #include "State.h"
20 using namespace std;
21 class RaffleActivity;
22 //能抽奖状态
23 class CanRaffleState :public State
24 {
25 public:
26     CanRaffleState(RaffleActivity* activity);
27     //已经扣除积分了, 不能再扣了
28     void deductMeney() override;
29     //可以抽奖, 根据抽奖情况改成新的状态
30     bool raffle() override;
31     void dispensePrize() override;
32 private:
33     RaffleActivity* activity;
34 };
35
36 //CanRaffleState.cpp
37 #include "CanRaffleState.h"
38 #include "RaffleActivity.h"
39 CanRaffleState::CanRaffleState(RaffleActivity* activity)
40 {
41

```

```

42     srand(time(NULL));
43     this->activity = activity;
44 }
45
46 //已经扣除积分了, 不能再扣了
47 void CanRaffleState::deductMeney()
48 {
49     cout << "已经扣过积分了" << endl;
50 }
51
52 //可以抽奖, 根据抽奖情况改成新的状态
53 bool CanRaffleState::raffle()
54 {
55     cout << "正在抽奖" << endl;
56     int result = rand() % 10;
57     if (0 == result)
58     {
59         //将activity的状态设置为发放奖品的状态
60         activity->setState(activity->getDispenseState());
61         return true;
62     }
63     else
64     {
65         cout << "很遗憾没有抽中奖品" << endl;
66         //将activity的状态设置为不能抽奖的状态
67         activity->setState(activity->getNoRaffleState());
68         return false;
69     }
70 }
71
72 void CanRaffleState::dispensePrize()
73 {
74     cout << "没中奖, 不能发放奖品" << endl;
75 }
76
77 //RaffleActivity.h
78 #pragma once
79 #include "State.h"
80 class RaffleActivity;
81 //奖品发送完毕状态
82 class DispenseOutState :public State
83 {
84 public:
85     DispenseOutState(RaffleActivity* activity);
86     void deductMeney() override;
87     bool raffle() override;
88     //发放奖品
89     void dispensePrize() override;
90
91 private:
92     RaffleActivity* activity;
93 };
94
95 //RaffleActivity.cpp
96 #include "DispenseOutState.h"
97 #include "RaffleActivity.h"
98
99 DispenseOutState::DispenseOutState(RaffleActivity* activity)
100 {
101     this->activity = activity;
102 }
103
104 void DispenseOutState::deductMeney()
105 {
106     cout << "奖品已经发完了, 请下次再参加" << endl;
107 }
108
109 bool DispenseOutState::raffle()
110 {
111     cout << "奖品已经发完了, 请下次再参加" << endl;
112     return false;

```

```

113 }
114
115 //发放奖品
116 void DispenseOutState::dispensePrize()
117 {
118     cout << "奖品已经发完了, 请下次再参加" << endl;
119 }
120 //RaffleActivity.h
121 #pragma once
122 #include <iostream>
123 #include "State.h"
124
125 using namespace std;
126
127 class RaffleActivity;
128
129 //发放奖品的状态
130 class DispenseState :public State
131 {
132 public:
133     DispenseState(RaffleActivity* activity);
134     void deductMeney() override;
135     bool raffle() override;
136     //发放奖品
137     void dispensePrize() override;
138
139 private:
140     RaffleActivity* activity;
141 };
142 //DispenseState.cpp
143 #include "DispenseState.h"
144 #include "RaffleActivity.h"
145
146 DispenseState::DispenseState(RaffleActivity* activity)
147 {
148     this->activity = activity;
149 }
150
151 void DispenseState::deductMeney()
152 {
153     cout << "不能扣除积分" << endl;
154 }
155
156 bool DispenseState::raffle()
157 {
158     cout << "不能抽奖了" << endl;
159     return false;
160 }
161
162 //发放奖品
163 void DispenseState::dispensePrize()
164 {
165     if (activity->getCount() > 0)
166     {
167         cout << "送出奖品" << endl;
168         activity->setState(activity->getNoRaffleState());
169     }
170     else
171     {
172         cout << "很遗憾, 奖品发完了" << endl;
173         //奖品已经发完, 后面的人就不能抽奖了
174         activity->setState(activity->getDispenseOutState());
175     }
176 }
177 //RaffleActivity.h
178 #pragma once
179 #include <iostream>
180 #include "State.h"
181
182 using namespace std;
183 class RaffleActivity;
184

```

```

185 //不能抽奖状态
186 class NoRaffleState :public State
187 {
188 public:
189     NoRaffleState(RaffleActivity* activity);
190     //在不能抽奖状态是可以扣积分的, 扣除积分后设置成可以抽奖状态
191     void deductMeney() override;
192     bool raffle() override;
193     void dispensePrize() override;
194
195 private:
196     //初始化时传入活动, 扣除积分后改变其状态
197     RaffleActivity* activity;
198 };
199
200 //NoRaffleState.cpp
201 #include "NoRaffleState.h"
202 #include "RaffleActivity.h";
203
204 NoRaffleState::NoRaffleState(RaffleActivity* activity)
205 {
206     this->activity = activity;
207 }
208
209 //在不能抽奖状态是可以扣积分的, 扣除积分后设置成可以抽奖状态
210 void NoRaffleState::deductMeney()
211 {
212     std::cout << "扣除50积分, 可以抽奖了" << endl;
213     activity->setState(activity->getCanRaffleState());
214 }
215
216 bool NoRaffleState::raffle()
217 {
218     cout << "扣了积分才能抽奖" << endl;
219     return false;
220 }
221
222 void NoRaffleState::dispensePrize()
223 {
224     cout << "不能发奖品" << endl;
225 }
226
227 //RaffleActivity.h
228 #pragma once
229 #include "CanRaffleState.h"
230 #include "DispenseOutState.h"
231 #include "DispenseState.h"
232 #include "State.h"
233 #include "NoRaffleState.h"
234
235 class RaffleActivity
236 {
237 public:
238     State* getState() const{
239         return state;
240     }
241     void setState(State* const state)
242     {
243         this->state = state;
244     }
245
246     int getCount()
247     {
248         return count--;
249     }
250
251     void setCount(const int count)
252     {
253         this->count = count;
254     }
255

```

```

256
257     State* getNoRaffleState() const
258     {
259         return noRaffleState;
260     }
261
262     void setNoRaffleState(State* const noRaffleState)
263     {
264         this->noRaffleState = noRaffleState;
265     }
266
267     State* getCanRaffleState() const
268     {
269         return canRaffleState;
270     }
271
272     void setCanRaffleState(State* const canRaffleState)
273     {
274         this->canRaffleState = canRaffleState;
275     }
276
277     State* getDispenseState() const
278     {
279         return dispenseState;
280     }
281
282     void setDispenseState(State* const dispenseState)
283     {
284         this->dispenseState = dispenseState;
285     }
286
287     State* getDispenseOutState() const
288     {
289         return dispenseOutState;
290     }
291
292     void setDispenseOutState(State* const dispenseOutState)
293     {
294         this->dispenseOutState = dispenseOutState;
295     }
296
297     RaffleActivity(int count)
298     {
299         //初始化当前状态为 不能抽奖状态
300         this->state = getNoRaffleState();
301         //初始化奖品数量
302         this->count = count;
303     }
304
305     //扣分, 调用当前状态的deductMoney
306     void deductMoney()
307     {
308         state->deductMoney();
309     }
310
311     //抽奖
312     void raffle()
313     {
314         //如果抽中奖了, 则领奖品
315         if (state->raffle())
316         {
317             state->dispensePrize();
318         }
319     }
320
321 private:
322     //state表示活动当前的状态
323     State* state = nullptr;
324     //奖品数量
325     int count = 0;
326     //四个属性 表示四种状态
327

```

```

327     State* noRaffleState = new NoRaffleState(this);
328     State* canRaffleState = new CanRaffleState(this);
329     State* dispenseState = new DispenseState(this);
330     State* dispenseOutState = new DispenseOutState(this);
331 };
332
333 //main.cpp
334 #include <iostream>
335 #include "RaffleActivity.h"
336 using namespace std;
337 int main(int argc, char* argv[])
338 {
339     RaffleActivity* activity = new RaffleActivity(1);
340
341     for(int i=0;i<50;i++)
342     {
343         cout << "第" << i << "次抽奖" << endl;
344         activity->deductMoney();
345         activity->raffle();
346         cout << endl;
347     }
348     return 0;
349 }

```

优点:

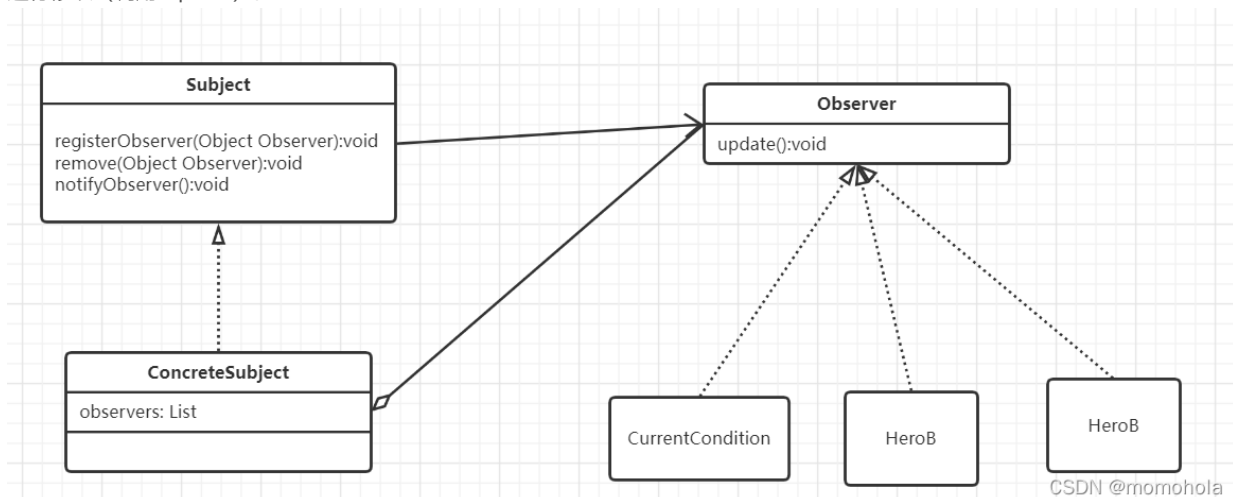
1. 封装了转换规则。
2. 枚举可能的状态，在枚举状态之前需要确定状态种类。
3. 将所有与某个状态有关的行为放到一个类中，并且可以方便地增加新的状态，只需要改变对象状态即可改变对象的行为。
4. 允许状态转换逻辑与状态对象合成一体，而不是某一个巨大的条件语句块。
5. 可以让多个环境对象共享一个状态对象，从而减少系统中对象的个数。

Observer 模式(观察者模式)

Observer 模式要解决的问题为：建立一个一（Subject）对多（Observer）的依赖关系，并且做到当“一”变化的时候，依赖这个“一”的多也能够同步改变。最常见的一个例子就是：对同一组数据进行统计分析时候，我们希望能够提供多种形式的表示（例如以表格进行统计显示、柱状图统计显示、百分比统计显示等）。指多个对象间存在一对多的依赖关系，当一个对象的状态发生改变时，所有依赖于它的对象都得到通知并被自动更新。

举例:

Subject 提供依赖于它的观察者 Observer 的注册（registerObserver）和注销（remove）操作，并且提供了使得依赖于它的所有观察者同步的操作（notifyObserver），观察者 Observer 则提供一个 Update 操作，注意这里的 Observer 的 Update 操作并不在 Observer 改变了 Subject 目标状态的时候就对自己进行更新，这个更新操作要延迟到 Subject 对象发出 Notify 通知所有 Observer 进行修改（调用 Update）。



```

1 #include <iostream>
2 #include <list>
3 using namespace std;
4

```

```

5
6 //抽象的英雄 抽象的观察者 Observer
7 class AbstractHero {
8 public:
9     virtual void Update() = 0;
10 };
11
12 //具体的英雄 具体的观察者
13 class HeroA :public AbstractHero {
14 public:
15     HeroA() {
16         cout << "英雄A正在鲁BOSS" << endl;
17     }
18     virtual void Update() {
19         cout << "英雄A停止撸，待机状态" << endl;
20     }
21 };
22
23 class HeroB :public AbstractHero {
24 public:
25     HeroB() {
26         cout << "英雄B正在鲁BOSS" << endl;
27     }
28     virtual void Update() {
29         cout << "英雄B停止撸，待机状态" << endl;
30     }
31 };
32
33 class HeroC :public AbstractHero {
34 public:
35     HeroC() {
36         cout << "英雄C正在鲁BOSS" << endl;
37     }
38     virtual void Update() {
39         cout << "英雄C停止撸，待机状态" << endl;
40     }
41 };
42
43 class HeroD :public AbstractHero {
44 public:
45     HeroD() {
46         cout << "英雄D正在鲁BOSS" << endl;
47     }
48     virtual void Update() {
49         cout << "英雄D停止撸，待机状态" << endl;
50     }
51 };
52
53 class HeroE :public AbstractHero {
54 public:
55     HeroE() {
56         cout << "英雄E正在鲁BOSS" << endl;
57     }
58     virtual void Update() {
59         cout << "英雄E停止撸，待机状态" << endl;
60     }
61 };
62
63 //定义抽象的观察目标 Subject
64 class AbstractBoss {
65 public:
66     // 添加观察者
67     virtual void addHero(AbstractHero* hero) = 0;
68     // 删除观察者
69     virtual void deleteHero(AbstractHero* hero) = 0;
70     // 通知所有观察者
71     virtual void notifi() = 0;
72 };
73
74 //相对于 concreteSubject
75

```

```

76 class BOSSA :public AbstractBoss {
77 public:
78     //    添加观察者
79     virtual void addHero(AbstractHero* hero) {
80         pHeroList.push_back(hero);
81     }
82     //    删除观察者
83     virtual void deleteHero(AbstractHero* hero) {
84         pHeroList.remove(hero);
85     }
86     //    通知所有观察者
87     virtual void notifv() {
88         for (list<AbstractHero*>::iterator it = pHeroList.begin(); it != pHeroList.end(); it++) {
89             (*it)->Update();
90         }
91     }
92
93     list<AbstractHero*> pHeroList;
94 };
95
96 void test01() {
97     //    创建观察者
98     AbstractHero* heroA = new HeroA;
99     AbstractHero* heroB = new HeroB;
100    AbstractHero* heroC = new HeroC;
101    AbstractHero* heroD = new HeroD;
102    AbstractHero* heroE = new HeroE;
103
104    //    创建观察目标
105    AbstractBoss* bossA = new BOSSA;
106    bossA->addHero(heroA);
107    bossA->addHero(heroB);
108    bossA->addHero(heroC);
109    bossA->addHero(heroD);
110    bossA->addHero(heroE);
111
112    cout << "heroC阵亡" << endl;
113    bossA->deleteHero(heroC);
114
115    cout << "Boss死了，通知其他英雄停止攻击。。。" << endl;
116    bossA->notifv();
117
118 }
119
120 int main() {
121
122     test01();
123     return 0;
124 }

```

优点:

1. 观察者和被观察者是抽象耦合的。
2. 建立一套触发机制。

缺点:

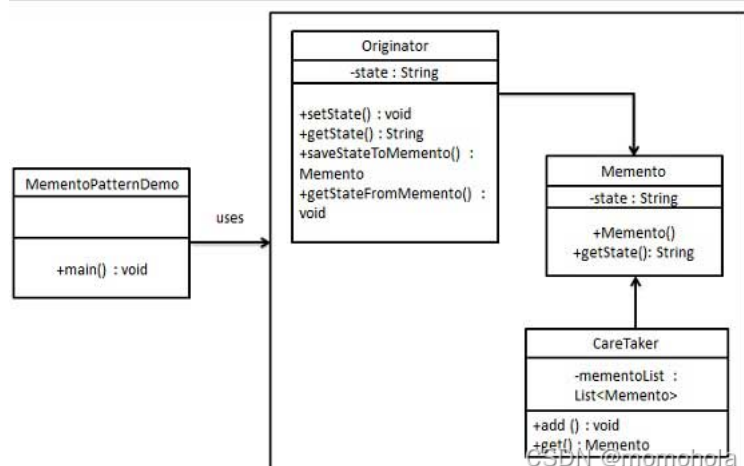
1. 如果在观察者和观察目标之间有循环依赖的话，观察目标会触发它们之间进行循环调用，可能导致系统崩溃。
2. 观察者模式没有相应的机制让观察者知道所观察的目标对象是怎么发生变化的，而仅仅只是知道观察目标发生了变化。

Memento 模式（备忘录模式）

没有人想犯错误，但是没有人能够不犯错误。犯了错误一般只能改过，却很难改正（恢复）。世界上没有后悔药，但是我们在进行软件系统的设计时候是要给用户后悔的权利（实际上可能也是用户要求的权利：）），我们对一些关键性的操作肯定需要提供诸如撤销（Undo）的操作。那这个后悔药就是 Memento 模式提供的。

Memento 模式的关键就是要在不破坏封装行的前提下，捕获并保存一个类的内部状态，这样就可以利用该保存的状态实施恢复操作。

示例：



代码：

- Originator 原始的状态
- Memento 包含了要被恢复的对象的状态
- Caretaker 用于保存所有的memento的一个管理类

我们有一个英雄，有一个初始状态，在收到敌人的DEBUFF后状态会下降，在DEBUFF时间过了之后又会回到正常的状态，所以我们需要保存正常状态的值，这个时候我们就可以采用备忘录模式了。

```
1 //Originator.h
2 #ifndef _ORIGINATOR_H_
3 #define _ORIGINATOR_H_
4 #include <string>
5
6 #include "Memento.h"
7 class Originator
8 {
9 public:
10     std::string getState() const
11     {
12         return state;
13     }
14
15     void setState(const std::string& state)
16     {
17         this->state = state;
18     }
19
20     //保存一个状态对象Memento 即用当前状态的值得去创建一个Memento然后将其返回
21     Memento SaveStateMemento()
22     {
23         return Memento(state);
24     }
25
26     //通过备忘录对象，恢复状态
27     void getStateFromMemento(Memento memento)
28     {
29         state = memento.getState();
30     }
31 private:
32     std::string state;
33 };
34 #endif
35
36 //Memento.h
37 #ifndef _MEMENTO_H_
38 #define _MEMENTO_H_
39 #include <string>
```

```

40 class Memento
41 {
42 public:
43     explicit Memento(const std::string& state)
44         : state(state)
45     {
46     }
47
48     std::string getState() const
49     {
50         return state;
51     }
52
53 private:
54     std::string state;
55 };
56
57 #endif
58
59 //Caretaker.h
60 #ifndef _CARETAKER_H_
61 #define _CARETAKER_H_
62
63 #include <vector>
64 #include "Memento.h"
65 class Caretaker
66 {
67 public:
68     void add(Memento memento)
69     {
70         mementoList.push_back(memento);
71     }
72
73     //获取到第index个Originator的备忘录对象 (即备忘录状态)
74     Memento get(int index)
75     {
76         return mementoList[index];
77     }
78 private:
79     //在mementoList中会有很多备忘录对象
80     std::vector<Memento> mementoList;
81 };
82 #endif
83
84 //main.h
85 #include <iostream>
86 #include "Originator.h"
87 #include "Caretaker.h"
88 using namespace std;
89 int main(int argc, char* argv[])
90 {
91     Originator originator;
92     Caretaker caretaker;
93     originator.setState("状态1, 攻击力为100");
94
95     //保存当前状态
96     caretaker.add(originator.SaveStateMemento());
97
98     //受到debuff, 攻击力下降
99     originator.setState("状态2, 攻击力为80");
100     //保存状态
101     caretaker.add(originator.SaveStateMemento());
102
103     cout << "当前的状态: " << originator.getState()<<endl;
104     cout << "debuff时间结束, 回到状态1" << endl;
105     originator.getStateFromMemento(caretaker.get(0));
106     cout << "恢复到状态1后的状态: " << originator.getState();
107     return 0;
108 }

```

总结:

优点:

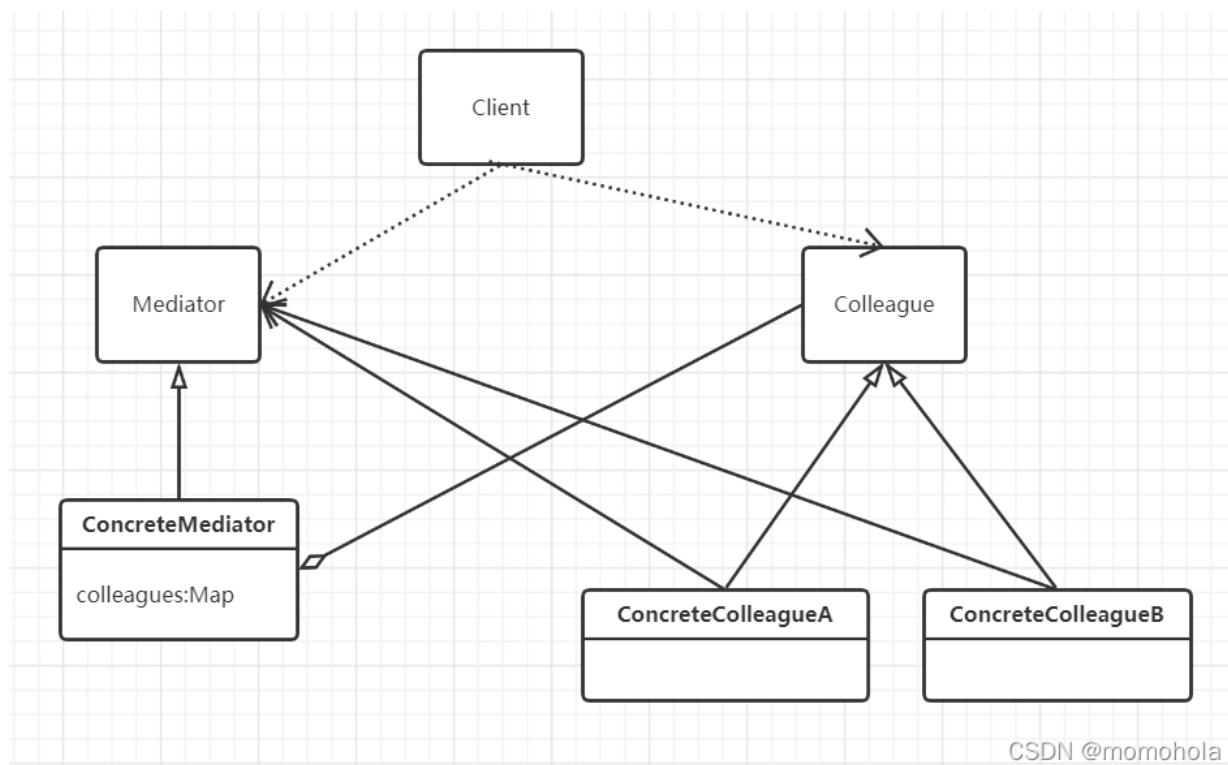
1. 给用户提供了一种可以恢复状态的机制，可以使用户能够比较方便地回到某个历史的状态。
2. 实现了信息的封装，使得用户不需要关心状态的保存细节。

缺点: 消耗资源。如果类的成员变量过多，势必会占用比较大的资源，而且每一次保存都会消耗一定的内存。

Mediator 模式 (中介者模式)

在面向对象系统的设计和开发过程中，对象之间的交互和通信是最为常见的情况，因为对象间的交互本身就是一种通信。在系统比较小的时候，可能对象间的通信不是很多、对象也比较少，我们可以直接硬编码到各个对象的方法中。但是当系统规模变大，对象的量变引起系统复杂度的急剧增加，对象间的通信也变得越来越复杂，这时候我们就要提供一个专门处理对象间交互和通信的类，这个中介者就是 Mediator 模式。所以 Mediator 模式的实现关键就是将对象 Colleague 之间的通信封装到一个类种单独处理。

举例:

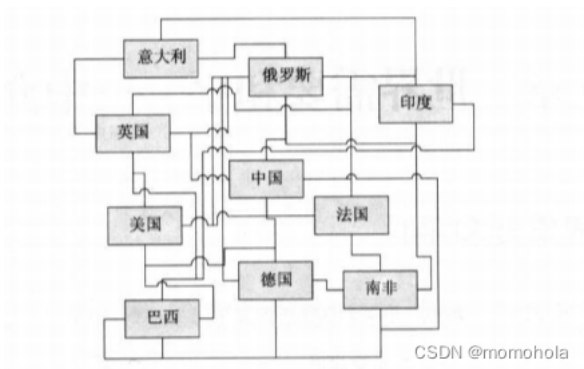


说明:

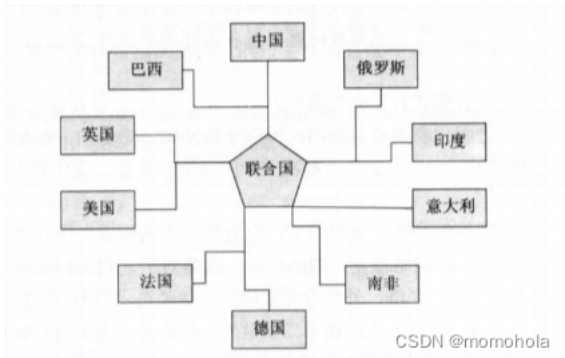
1. Mediator是抽象中介者，定义了同事对象到中介者对象的接口。
2. Colleague是抽象同事类
3. ConcreteMediator是具体的中介者对象，实现抽象方法，他需要知道所有的具体的同事类，用一个集合来管理它们，并接收某个同事的消息，完成相应的任务。
4. ConcreteColleague是具体的同事类，会有很多，每个同事只知道自己的行为，而不了解其他同事的行为，它们都依赖中介者对象。

中介者模式的目标:

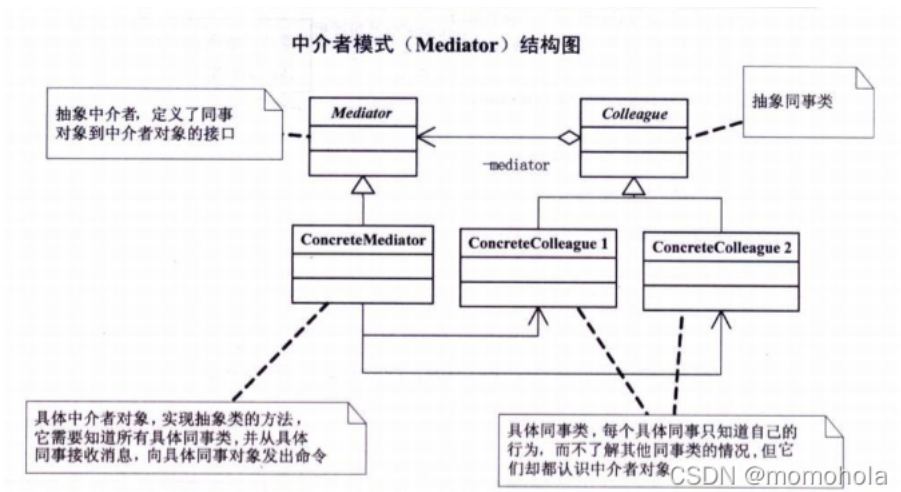
将网状结构改成星状结构



使用中介者模式后



举例:



代码:

```

1 //Colleague.h
2 #pragma once
3 #include <string>
4
5 class Mediator;
6
7 class Colleague
8 {
9 public:
10     Mediator* getMediator();
11     void setMediator(Mediator* const mediator);
12     Colleague(Mediator* mediator);
13     virtual void Notify(std::string message)=0;
14
15 private:
16     Mediator* mediator;
17 };
18 //Colleague.cpp
19 #include "Colleague.h"
20 #include "Mediator.h"
21
22 Mediator* Colleague::getMediator()
23 {
24

```

```

24     return mediator;
25 }
26
27
28 void Colleague::setMediator(Mediator* const mediator)
29 {
30     this->mediator = mediator;
31 }
32
33 Colleague::Colleague(Mediator* mediator)
34 {
35     this->mediator = mediator;
36     this->mediator->add(this);
37 }
38
39 //ConcreteColleague1.h
40 #ifndef _CONCRETECOLLEAGUE1_H_
41 #define _CONCRETECOLLEAGUE1_H_
42 #include <iostream>
43 #include <string>
44 #include "Colleague.h"
45 #include "Mediator.h"
46 class ConcreteColleague1:public Colleague
47 {
48 public:
49     ConcreteColleague1(Mediator* mediator):Colleague(mediator)
50     {
51     }
52
53     void send(std::string message)
54     {
55         getMediator()->send(message, this);
56     }
57
58     void Notify(std::string message)
59     {
60         std::cout << "同事1收到消息: " + message<<std::endl;
61     }
62 };
63 #endif
64
65 //ConcreteColleague2.h
66 #ifndef _CONCRETECOLLEAGUE2_H_
67 #define _CONCRETECOLLEAGUE2_H_
68 #include <iostream>
69 #include <string>
70 #include "Colleague.h"
71 #include "Mediator.h"
72
73 class ConcreteColleague2 :public Colleague
74 {
75 public:
76     ConcreteColleague2(Mediator* mediator) :Colleague(mediator)
77     {
78     }
79
80     void send(std::string message)
81     {
82         getMediator()->send(message, this);
83     }
84
85     void Notify(std::string message)
86     {
87         std::cout << "同事2收到消息: " + message << std::endl;
88     }
89 };
90 #endif
91
92 //ConcreteMediator.h
93 #ifndef _CONCRETEMEDIATOR_H_
94 #define _CONCRETEMEDIATOR_H_
95 #include <vector>

```

```

96 #include "Colleague.h"
97 #include "Mediator.h"
98 class ConcreteMediator:public Mediator
99 {
100 public:
101     void add(Colleague* colleague)
102     {
103         colleaguesList.push_back(colleague);
104     }
105
106     void send(std::string message, Colleague* colleague)
107     {
108         for (auto value : colleaguesList)
109         {
110             if(value!=colleague)
111             {
112                 value->Notify(message);
113             }
114         }
115     }
116 private:
117     std::vector<Colleague*> colleaguesList;
118 };
119 #endif
120
121 //Mediator.h
122 #ifndef _MEDIATOR_H_
123 #define _MEDIATOR_H_
124 #include <string>
125 #include "Colleague.h"
126 class Mediator
127 {
128 public:
129     virtual void send(std::string message, Colleague* colleague)=0;
130     virtual void add(Colleague* colleague)=0;
131 };
132 #endif
133
134 //main.cpp
135 #include <iostream>
136 #include "ConcreteMediator.h"
137 #include "ConcreteColleague1.h"
138 #include "ConcreteColleague2.h"
139 #include "Mediator.h"
140 #include "Colleague.h"
141
142 using namespace std;
143 int main(int argc, char* argv[])
144 {
145     Mediator* mediator = new ConcreteMediator();
146     ConcreteColleague1* colleague1 = new ConcreteColleague1(mediator);
147     ConcreteColleague2* colleague2 = new ConcreteColleague2(mediator);
148
149     colleague1->send("早上好啊! ");
150     colleague2->send("早安! ");
151     return 0;
152 }

```

总结:

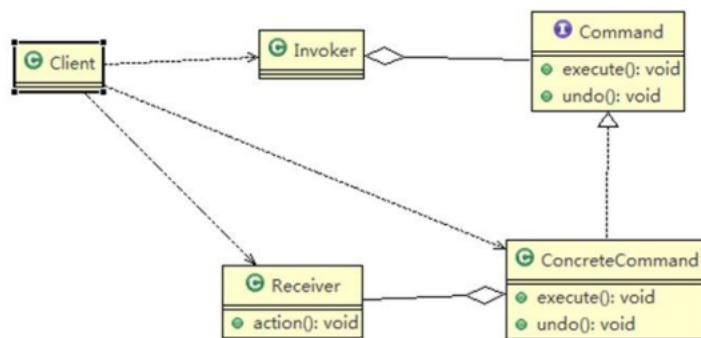
- **何时使用:** 多个类相互耦合，形成了网状结构，需要将上述网状结构分离为星型结构的时候。
- 中介者承担了较多的责任，一旦中介者出现问题，整个系统都会收到影响。

Command 模式 (命令模式)

- 请求以命令的形式包裹在对象中，并传给调用对象。调用对象寻找可以处理该命令的合适的对象，并把该命令传给相应的对象，该对象执行命令。

- Command 模式的最终目的就是将一个请求封装成一个对象，从而使您可以用不同的请求对客户进行参数化。

举例：



CSDN @momohola

- Invoker 是调用者角色
- Command 是命令角色，需要执行的所有命令都在这里
- Receiver 是接受者角色，知道如何实施和执行一个请求相关的操作
- ConcreteCommand 将接受者对象与一个动作绑定，调用接受者相应的操作

假如现在有一个游戏服务器，该游戏服务器一共可以处理四种不同的请求：处理增加金币、处理增加钻石、处理玩家装备、玩家升级请求。我们需要把这些请求封装成对象，从而加入请求队列一次进行处理。

代码：

```

1 //命令模式
2 #include <iostream>
3 #include <queue>
4 #include <Windows.h>
5 using namespace std;
6
7 //协议处理类 (请求类)
8 class HandleClnetProtocal
9 {
10 public:
11     //处理增加金币
12     void AddMoney()
13     {
14         cout << "给玩家增加金币" << endl;
15     }
16     //处理增加钻石
17     void AddDiamond()
18     {
19         cout << "给玩家增加钻石" << endl;
20     }
21     //处理玩家装备
22     void AddEquipment()
23     {
24         cout << "给玩家穿装备" << endl;
25     }
26     //玩家升级
27     void AddLevel()
28     {
29         cout << "给玩家升级" << endl;
30     }
31 };
32
33 //命令接口
34 class AbstractCommand
35 {
36 public:
37     //相当于execute ()
38     virtual void handle() = 0;
39 };
40
  
```

```

41
42
43 //下面是把每个功能都封装为一个请求对象
44 //处理增加金币请求
45 class AddMoneyCommand:public AbstractCommand
46 {
47 public:
48     AddMoneyCommand(HandleClnetProtocal* protocal)
49     {
50         this->pProtocol = protocal;
51     }
52     virtual void handle()
53     {
54         this->pProtocol->AddMoney();
55     }
56 public:
57     HandleClnetProtocal* pProtocol;
58 };
59
60 //处理增加钻石请求
61 class AddDiamondCommand :public AbstractCommand
62 {
63 public:
64     AddDiamondCommand(HandleClnetProtocal* protocal)
65     {
66         this->pProtocol = protocal;
67     }
68     virtual void handle()
69     {
70         this->pProtocol->AddDiamond();
71     }
72 public:
73     HandleClnetProtocal* pProtocol;
74 };
75
76 //处理玩家装备请求
77 class AddEquipmentCommand :public AbstractCommand
78 {
79 public:
80     AddEquipmentCommand(HandleClnetProtocal* protocal)
81     {
82         this->pProtocol = protocal;
83     }
84     virtual void handle()
85     {
86         this->pProtocol->AddEquipment();
87     }
88 public:
89     HandleClnetProtocal* pProtocol;
90 };
91
92 //处理玩家升级请求
93 class AddLevelCommand :public AbstractCommand
94 {
95 public:
96     AddLevelCommand(HandleClnetProtocal* protocal)
97     {
98         this->pProtocol = protocal;
99     }
100     virtual void handle()
101     {
102         this->pProtocol->AddLevel();
103     }
104 public:
105     HandleClnetProtocal* pProtocol;
106 };
107
108 //服务器程序    (命令调用类)
109 class Server
110 {
111 public:

```



```

112 //将请求对象放入处理队列
113 void addRequest(AbstractCommand* command)
114 {
115     mCommands.push(command);
116 }
117
118 //启动处理程序
119 void startHandle()
120 {
121     while (!mCommands.empty())
122     {
123         Sleep(2000);
124         AbstractCommand* command = mCommands.front();
125         command->handle();
126         mCommands.pop();
127     }
128 }
129 queue<AbstractCommand*> mCommands;
130 };
131
132 void test01()
133 {
134     HandleClnetProtocal* protocal = new HandleClnetProtocal;
135     //客户端增加金币的请求
136     AbstractCommand* addmoney = new AddMoneyCommand(protocal);
137     //客户端增加钻石的请求
138     AbstractCommand* adddiamond = new AddDiamondCommand(protocal);
139     //客户端穿装备的请求
140     AbstractCommand* addequipment = new AddEquipmentCommand(protocal);
141     //客户端升级请求
142     AbstractCommand* addlevel = new AddLevelCommand(protocal);
143
144     //将客户端的请求加入到请求队列中
145     Server* server = new Server;
146     server->addRequest(addmoney);
147     server->addRequest(adddiamond);
148     server->addRequest(addequipment);
149     server->addRequest(addlevel);
150
151     //服务器开始处理请求
152     server->startHandle();
153 }
154
155
156 int main()
157 {
158     test01();
159     return 0;
160 }

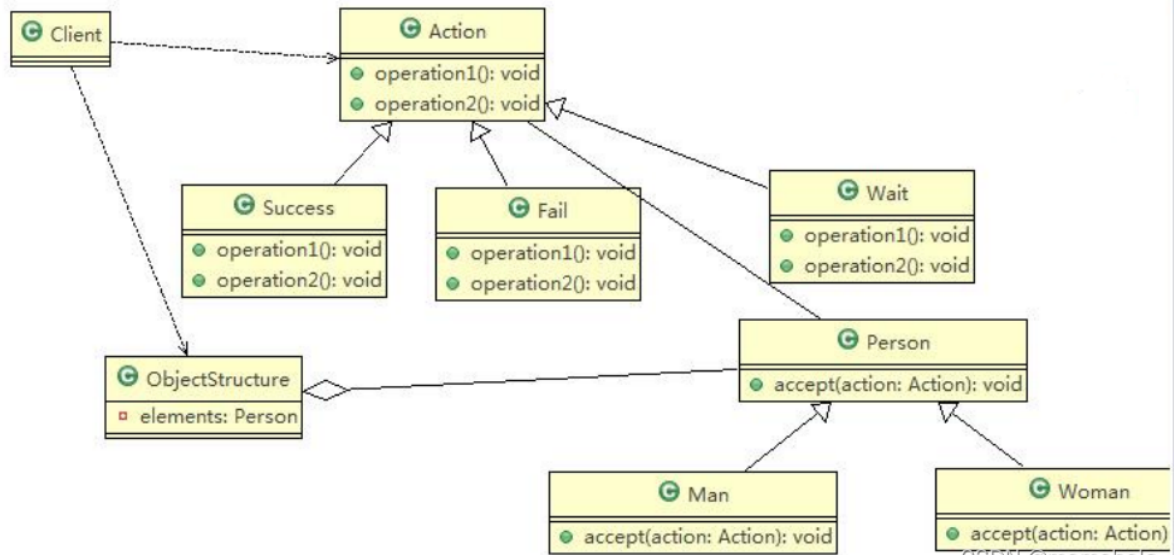
```

Visitor 模式（访问者模式）

- 访问者模式（VisitorPattern），封装一些作用于某种数据结构的各元素的操作，它可以在不改变数据结构的前提下定义作用于这些元素的新的操作。
- 主要将数据结构与数据操作分离，解决 数据结构和操作耦合性问题
- 访问者模式的基本工作原理是：在被访问的类里面加一个对外提供接待访问者的接口
- 访问者模式主要应用场景是：需要对一个对象结构中的对象进行很多不同操作(这些操作彼此没有关联)，同时需要避免让这些操作“污染”这些对象的类，可以选用访问者模式解决

举例：

现在有一场演出，将参赛选手分为男人和女人，当看完某个歌手表演后，需要对他们进行评价（失败、成功等）



CSDN @momohola

代码:

```

1  #include <iostream>
2  #include <list>
3
4  class Man;
5  class Woman;
6  using namespace std;
7
8  class Action
9  {
10 public:
11     virtual void getManResult(Man* man) =0;
12     virtual void getWomanResult(Woman* woman) =0;
13 private:
14 };
15
16
17 class Success:public Action
18 {
19 public:
20     void getManResult(Man* man) override
21     {
22         cout << "男人的给的评价该歌手是很成功" << endl;
23     }
24
25     void getWomanResult(Woman* woman) override
26     {
27         cout << "女人的给的评价该歌手是很成功" << endl;
28     }
29 };
30
31 class Fail :public Action
32 {
33 public:
34     void getManResult(Man* man) override
35     {
36         cout << "男人的给的评价该歌手是失败" << endl;
37     }
38
39     void getWomanResult(Woman* woman) override
40     {
41         cout << "女人的给的评价该歌手是失败" << endl;
42     }
43 };
44
45 class Person
46 {
47 public:
48     //提供一个方法让访问者可以访问

```

```

49     virtual void accept(Action* action) = 0;
50 };
51
52 //这里用到了双分派，即首先在客户端程序中，将具体的状态作为参数传递进Man中（第一次分派）
53 //然后在Man中调用作为参数的“具体方法” getManResult同时将自己作为参数传入完成第二次分派
54 class Man:public Person
55 {
56 public:
57     void accept(Action* action) override
58     {
59         action->getManResult(this);
60     }
61 };
62
63 class Woman: public Person
64 {
65 public:
66     void accept(Action* action) override
67     {
68         action->getWomanResult(this);
69     }
70 };
71
72 //数据结构，管理很多人（Man、Woman）用来装符合某一个评价的所有人
73 class ObjectStructure
74 {
75 public:
76     //添加到list
77     void attach(Person* p)
78     {
79         persons.push_back(p);
80     }
81     //移除
82     void detach(Person* p)
83     {
84         persons.remove(p);
85         delete p;
86     }
87
88     //显示测评情况
89     void display(Action* action)
90     {
91         for (auto value : persons)
92         {
93             value->accept(action);
94         }
95     }
96 private:
97     list<Person*> persons;
98 };
99
100 int main(int argc, char* argv[])
101 {
102     //创建ObjectStructure（可以创建很多个不同的ObjectStructure来代表不同评价，然后把同样评价的人放到一个ObjectStructure中）
103     ObjectStructure* objectStructure = new ObjectStructure;
104     objectStructure->attach(new Man);
105     objectStructure->attach(new Woman);
106
107     //假如歌手获得成功
108     Success* success = new Success;
109     objectStructure->display(success);
110     cout << "===== " << endl;
111     //假如歌手失败了
112     Fail* fail = new Fail;
113     objectStructure->display(fail);
114     return 0;
115 }
116

```

总结:

- 假设我们要添加一个Wait的状态类，考察Man类和Woman类的反应，由于使用了双分派，只需增加一个Action子类即可在客户端调用即可，不需要改动任何其他类的代码

优点:

1. 访问者模式符合单一职责原则、让程序具有优秀的扩展性、灵活性非常高
2. 访问者模式可以对功能进行统一，可以做报表、UI、拦截器与过滤器，适用于数据结构相对稳定的系统

缺点:

1. 具体元素对访问者公布细节，也就是说访问者关注了其他类的内部细节，这是迪米特法则所不建议的，这样造成了具体元素变更比较困难
2. 违背了依赖倒转原则。访问者依赖的是具体元素，而不是抽象元素
3. 因此，如果一个系统有比较稳定的数据结构，又有经常变化的功能需求，那么访问者模式就是比较合适的。

Chain of Responsibility 模式（责任链模式）

责任链模式（Chain of Responsibility Pattern）为请求创建了一个接收者对象的链。这种模式给予请求的类型，对请求的发送者和接收者进行解耦。这种类型的设计模式属于行为型模式。

在这种模式中，通常每个接收者都包含对另一个接收者的引用。如果一个对象不能处理该请求，那么它会把相同的请求传给下一个接收者，依此类推。

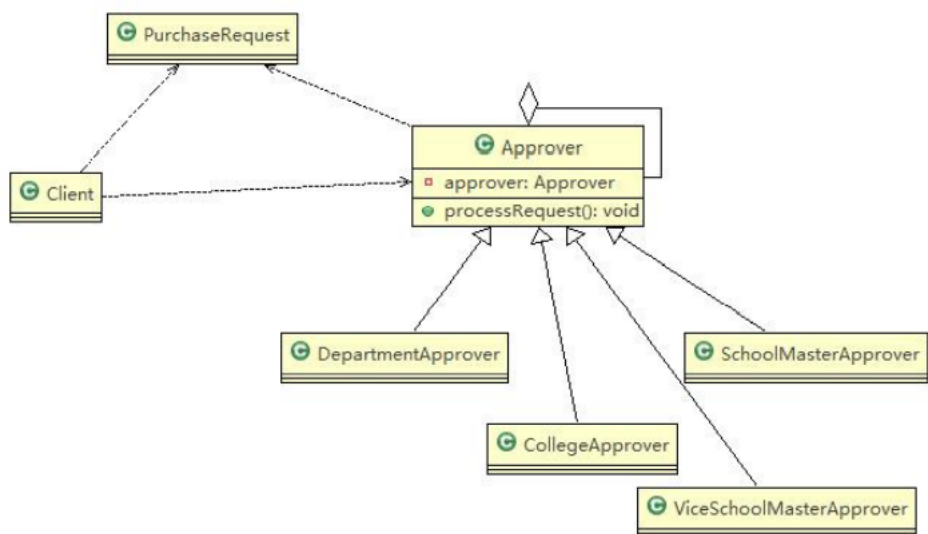
****主要解决：****职责链上的处理者负责处理请求，客户只需要将请求发送到职责链上即可，无须关心请求的处理细节和请求的传递，所以职责链将请求的发送者和请求的处理者解耦了。

****何时使用：****在处理消息的时候以过滤很多道。

举例:

编写程序完成学校OA系统的采购审批项目：需求：

- 采购员采购教学器材
- 如果金额 小于等于5000, 由教学主任审批
- 如果金额 小于等于10000, 由院长审批
- 如果金额 小于等于30000, 由副校长审批
- 如果金额 超过30000以上, 有校长审批



CSDN @momohola

代码:

```
1 | #include <iostream>
2 |
3 |
```

```

4   using namespace std;
5
6   //请求类
7   class PurchaseRequest
8   {
9   public:
10      int getType() const
11      {
12          return type;
13      }
14
15      float getPrice() const
16      {
17          return price;
18      }
19
20      int getId() const
21      {
22          return id;
23      }
24
25      PurchaseRequest(const int type, const float price, const int id)
26          : type(type),
27            price(price),
28            id(id)
29      {
30      }
31
32   private:
33      //请求类型
34      int type;
35      //价格
36      float price = 0.f;
37      int id = 0;
38  };
39
40  class Approver
41  {
42  public:
43
44      void setApprover(Approver* const approver)
45      {
46          this->approver = approver;
47      }
48
49      explicit Approver(const string& name)
50          : name(name)
51      {
52      }
53
54      //处理审批请求的方法，得到一个请求，处理是子类完成的，因此该方法写成纯虚方法
55      virtual void processRequest(PurchaseRequest* purchaseRequest)=0;
56  protected:
57      //下一个处理者
58      Approver* approver;
59      //名字
60      string name;
61  };
62  //教学主任
63  class DepartmentApprover:public Approver
64  {
65  public:
66      explicit DepartmentApprover(const string& name)
67          : Approver(name)
68      {
69      }
70
71      void processRequest(PurchaseRequest* purchaseRequest) override
72      {
73          if(purchaseRequest->getPrice()<=5000)
74          {

```

```

75         cout << "请求编号id=" << purchaseRequest->getId() << "被" << this->name << "处理" << endl;
76     }
77     else
78     {
79         approver->processRequest(purchaseRequest);
80     }
81 }
82 };
83 //院长
84 class CollegeApprover :public Approver
85 {
86 public:
87     explicit CollegeApprover(const string& name)
88         : Approver(name)
89     {
90     }
91
92     void processRequest(PurchaseRequest* purchaseRequest) override
93     {
94         if (purchaseRequest->getPrice() > 5000 && purchaseRequest->getPrice() <= 10000)
95         {
96             cout << "请求编号id=" << purchaseRequest->getId() << "被" << this->name << "处理" << endl;
97         }
98         else
99         {
100             approver->processRequest(purchaseRequest);
101         }
102     }
103 };
104 //副校长
105 class ViceSchoolMasterApprover :public Approver
106 {
107 public:
108     explicit ViceSchoolMasterApprover(const string& name)
109         : Approver(name)
110     {
111     }
112
113     void processRequest(PurchaseRequest* purchaseRequest) override
114     {
115         if (purchaseRequest->getPrice() > 10000 && purchaseRequest->getPrice() <= 30000)
116         {
117             cout << "请求编号id=" << purchaseRequest->getId() << "被" << this->name << "处理" << endl;
118         }
119         else
120         {
121             approver->processRequest(purchaseRequest);
122         }
123     }
124 };
125 //校长
126 class SchoolMasterApprover :public Approver
127 {
128 public:
129     explicit SchoolMasterApprover(const string& name)
130         : Approver(name)
131     {
132     }
133
134     void processRequest(PurchaseRequest* purchaseRequest) override
135     {
136         if (purchaseRequest->getPrice() > 30000)
137         {
138             cout << "请求编号id=" << purchaseRequest->getId() << "被" << this->name << "处理" << endl;
139         }
140         else
141         {
142             approver->processRequest(purchaseRequest);
143         }
144     }
145 };
146

```

```

147 int main(int argc, char* argv[])
148 {
149     //创建一个请求
150     PurchaseRequest* purchaseRequest = new PurchaseRequest(1, 1000, 1);
151
152     //创建相关的审批人
153     DepartmentApprover* department = new DepartmentApprover("张主任");
154     CollegeApprover* college = new CollegeApprover("李院长");
155     ViceSchoolMasterApprover* viceSchoolMaster = new ViceSchoolMasterApprover("王副校长");
156     SchoolMasterApprover* schoolMaster = new SchoolMasterApprover("佟校长");
157
158     //需要将各个审批级别的下一个人设置好(处理人构成一个环装就可以从任何一个人开始处理了)
159     //否则必须从第一个处理人开始
160     department->setApprover(college);
161     college->setApprover(viceSchoolMaster);
162     viceSchoolMaster->setApprover(schoolMaster);
163     schoolMaster->setApprover(department);
164     //开始处理请求
165     viceSchoolMaster->processRequest(purchaseRequest);
166
167     return 0;
168 }

```

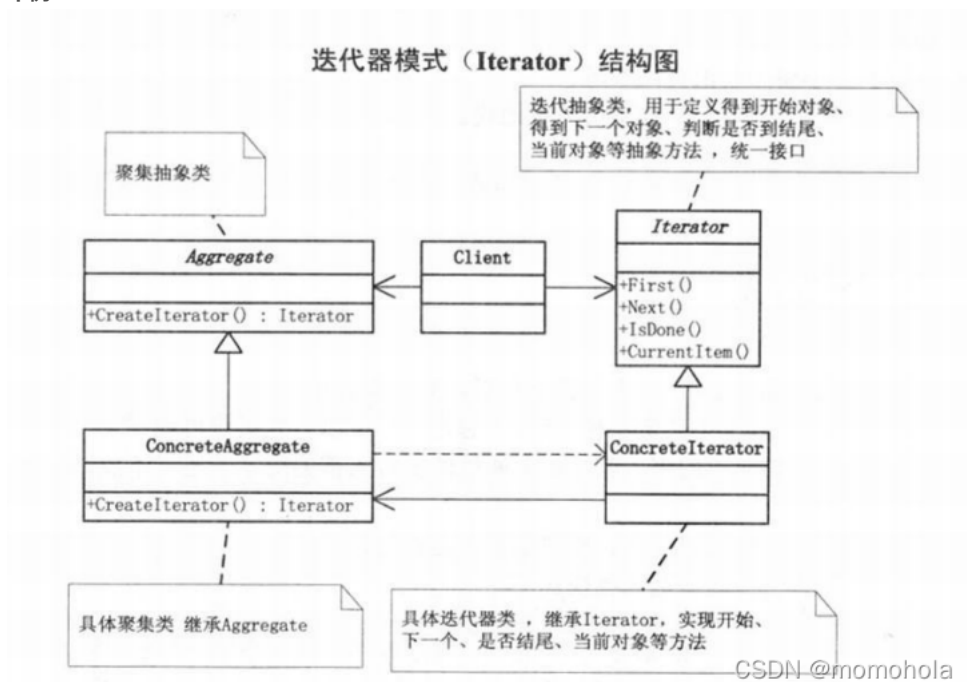
总结:

- 将请求和处理分开，实现解耦，提高系统的灵活性
- 简化了对象，使对象不需要知道链的结构
- 调试不方便。采用了类似递归的方式，调试时逻辑可能比较复杂
- 最佳应用场景：有多个对象可以处理同一个请求时，比如：多级请求、请假/加薪等审批流程、Java Web中Tomcat对Encoding的处理、拦截器

Iterator 模式 (迭代器模式)

迭代器模式，提供一种遍历集合元素的统一接口，用一致的方法遍历集合元素，不需要知道集合对象的底层表示，即：不暴露其内部的结构。而且不管这些对象是什么都需要遍历的时候，就应该选择使用迭代器模式

举例:



代码:

```

1 #include <iostream>
2 #include <string>
3 #include <vector>
4

```

```

4
5 using namespace std;
6
7 //迭代抽象类,用于定义得到开始对象、得到下一个对象、判断是否到结尾、当前对象等抽象方法,统一接口
8 class Iterator
9 {
10 {
11 public:
12     Iterator() {};
13     virtual ~Iterator() {};
14     virtual string First() = 0;
15     virtual string Next() = 0;
16     virtual string CurrentItem() = 0;
17     virtual bool IsDone() = 0;
18 };
19
20 //聚集抽象类
21 class Aggregate
22 {
23 public:
24     virtual int Count() = 0;
25     virtual void Push(const string& strValue) = 0;
26     virtual string Pop(const int nIndex) = 0;
27     virtual Iterator* CreateIterator() = 0;
28 };
29
30 //具体迭代器类,继承Iterator 实现开始、下一个、是否结尾、当前对象等方法
31 class ConcreteIterator : public Iterator
32 {
33 public:
34     ConcreteIterator(Aggregate* pAggregate) :m_nCurrent(0), Iterator()
35     {
36         m_Aggregate = pAggregate;
37     }
38     string First()
39     {
40         return m_Aggregate->Pop(0);
41     }
42     string Next()
43     {
44         string strRet;
45         m_nCurrent++;
46         if (m_nCurrent < m_Aggregate->Count())
47         {
48             strRet = m_Aggregate->Pop(m_nCurrent);
49         }
50         return strRet;
51     }
52     string CurrentItem()
53     {
54         return m_Aggregate->Pop(m_nCurrent);
55     }
56     bool IsDone()
57     {
58         return ((m_nCurrent >= m_Aggregate->Count()) ? true : false);
59     }
60 private:
61     Aggregate* m_Aggregate;
62     int m_nCurrent;
63 };
64
65 //具体聚集类 继承
66 class ConcreteAggregate : public Aggregate
67 {
68 public:
69     ConcreteAggregate() :m_pIterator(NULL)
70     {
71         m_vecItems.clear();
72     }
73     ~ConcreteAggregate()
74     {
75         if (NULL != m_pIterator)

```



```

76     {
77         delete m_pIterator;
78         m_pIterator = NULL;
79     }
80 }
81 Iterator* CreateIterator()
82 {
83     if (NULL == m_pIterator)
84     {
85         m_pIterator = new ConcreteIterator(this);
86     }
87     return m_pIterator;
88 }
89 int Count()
90 {
91     return m_vecItems.size();
92 }
93 void Push(const string& strValue)
94 {
95     m_vecItems.push_back(strValue);
96 }
97 string Pop(const int nIndex)
98 {
99     string strRet;
100    if (nIndex < Count())
101    {
102        strRet = m_vecItems[nIndex];
103    }
104    return strRet;
105 }
106 private:
107     vector<string> m_vecItems;
108     Iterator* m_pIterator;
109
110 };
111
112 int main()
113 {
114     ConcreteAggregate* pName = NULL;
115     pName = new ConcreteAggregate();
116     if (NULL != pName)
117     {
118         pName->Push("hello");
119         pName->Push("word");
120         pName->Push("cxue");
121     }
122     Iterator* iter = NULL;
123     iter = pName->CreateIterator();
124     if (NULL != iter)
125     {
126         string strItem = iter->First();
127         while (!iter->IsDone())
128         {
129             cout << iter->CurrentItem() << " is ok" << endl;
130             iter->Next();
131         }
132     }
133     system("pause");
134
135     return 0;
136 }

```

总结:

- 提供一个统一的方法遍历对象，客户不用再考虑聚合的类型，使用一种方法就可以遍历对象了。
- 隐藏了聚合的内部结构，客户端要遍历聚合的时候只能取到迭代器，而不会知道聚合的具体组成。
- 提供了一种设计思想，就是一个类应该只有一个引起变化的原因（叫做单一责任原则）。在聚合类中，我们把迭代器分开，就是要管理对象集合和遍历对象集合的责任分开，这样一来集合改变的话，只影响到聚合对象。而如果遍历方式改变的话，只影响到了迭代器。

- 当要展示一组相似对象，或者遍历一组相同对象时使用，适合使用迭代器模式

Interpreter 模式（解释器模式）

- 在编译原理中，一个算术表达式通过词法分析器形成词法单元，而后这些词法单元再通过语法分析器构建语法分析树，最终形成一颗抽象的语法分析树。这里的词法分析器和语法分析器都可以看做是解释器
- 解释器模式（Interpreter Pattern）：是指给定一个语言(表达式)，定义它的文法的一种表示，并定义一个解释器，使用该解释器来解释语言中的句子(表达式)
- 应用场景：
 - 1、应用可以将一个需要解释执行的语言中的句子表示为一个抽象语法树
 - 2、一些重复出现的问题可以用一种简单的语言来表达
 - 3、一个简单语法需要解释的场景
 - 4、这样的例子还有，比如编译器、运算表达式计算、正则表达式、机器人等

代码：

```
1  #include <iostream>
2  #include <string>
3
4  using namespace std;
5
6  class Context
7  {
8  public:
9      Context(int num)
10     {
11         m_num = num;
12     }
13
14 public:
15     void setNum(int num)
16     {
17         m_num = num;
18     }
19
20     int getNum()
21     {
22         return m_num;
23     }
24
25     void setRes(int res)
26     {
27         m_res = res;
28     }
29
30     int getRes()
31     {
32         return m_res;
33     }
34
35 private:
36     int m_num;
37     int m_res;
38 };
39
40 class Expression
41 {
42 public:
43     virtual void interpreter(Context *context) = 0;
44 };
45
46 class PlusExpression : public Expression
47 {
48 public:
49     virtual void interpreter(Context *context)
```

```

51     {
52         int num = context->getNum();
53         num++;
54         context->setNum(num);
55         context->setRes(num);
56     }
57 };
58
59 class MinusExpression : public Expression
60 {
61 public:
62     virtual void interpreter(Context *context)
63     {
64         int num = context->getNum();
65         num--;
66         context->setNum(num);
67         context->setRes(num);
68     }
69 };
70
71 int main(int argc, char **argv)
72 {
73     Context *pcxt = new Context(10);
74     Expression *e1 = new PlusExpression();
75
76     e1->interpreter(pcxt);
77     cout << "PlusExpression: " << pcxt->getRes() << endl;
78
79     Expression *e2 = new MinusExpression();
80
81     e2->interpreter(pcxt);
82     cout << "MinusExpression: " << pcxt->getRes() << endl;
83
84     delete e1;
85     delete e2;
86
87     delete pcxt;
88
89     return 0;
90 }

```

优点:

- 可扩展性比较好，灵活。
- 增加了新的解释表达式的方式。
- 易于实现简单文法。

缺点:

- 可利用场景比较少。
- 对于复杂的文法比较难维护。
- 解释器模式会引起类膨胀。
- 解释器模式采用递归调用方法。

引用

- 1)设计模式精解 - GoF 23 种设计模式解析附
- 2)菜鸟教程——设计模式
- 3)尚硅谷——设计模式
- 4)黑马——C++设计模式